Justify on Sui

Sui-based PolyMarkets, and the Sui bridge

A separate project

Native prediction markets in **Move** on Sui's object model, bridged to the EVM network via **Wormhole** — keeping the **Chainlink-CCIP-inspired** bridge architecture.

PolyMarket Social ("Justify") — Sui project documentation set

Justify on Sui — a separate project

Sui-based PolyMarkets, and the Sui bridge.

This is a **separate project** from the EVM line of Justify. Where the EVM work (see `documentation_bridge/`) deploys one Solidity market stack across Arc / Base / Ethereum / Polygon / BSC and bridges them with a Chainlink-CCIP-inspired layer, this project brings the same prediction-market product to **Sui** — a non-EVM, Move-language, object-centric chain — and then bridges Sui markets to the EVM world.

Two deliverables:

1. **Sui-based PolyMarkets** — the prediction-market stack rebuilt natively in **Move** on Sui's object model (not a Solidity port).
2. **The Sui bridge** — connecting Sui markets to the EVM home/proxy network, using **Wormhole** as the concrete cross-chain transport, while keeping the same **Chainlink-CCIP-inspired architectural layering** (adapter / proxy / home market, RMN-style risk controls) as the EVM bridge set.

> ***On Chainlink and Wormhole.*** *Sui is non-EVM and has no native Solidity or mature Chainlink CCIP lane today. So this project keeps the **architecture** inspired by Chainlink CCIP — the same separation of a chain-local router, a per-route adapter seam, a message-carrying transport, and an independent risk layer — but uses **Wormhole** (the established Sui↔EVM general-message bridge) as the concrete transport. The bridge stays transport-agnostic behind a Move adapter interface, so a Sui CCIP lane can be slotted in later without redesign.*

Reading order

#	Document	What it covers
00	sui-polymarkets-overview.md	Why Sui; the object-centric prediction-market model; product parity with the EVM stack
01	move-market-contracts.md	The Move package: factory, AMM, outcome coins, oracle — object-model design
02	sui-bridge-overview.md	The Sui bridge: home/proxy model on Sui, Chainlink-CCIP-inspired layering, Wormhole transport
03	wormhole-adapter-and-interop.md	The Wormhole adapter (Move + EVM sides), VAAs, Sui↔EVM message and value flow, wrapped receipts
04	sui-bridge-security.md	Security & risk; Wormhole Guardians vs Chainlink RMN; exposure caps, circuit breaker

Status

Designed (forward-looking), separate project. Nothing here is deployed. These documents define the target so a Move package and a Wormhole adapter can be built against a frozen seam, the same way `documentation_bridge/` does for EVM.

Relation to the rest of the repo

- **`documentation_bridge/`** — the EVM bridge set this project mirrors in architecture (home/proxy, `IBridgeAdapter`, Chainlink-CCIP-inspired layering).
- **`documentation_justify_whitepaper/`** — the protocol thesis; Sui extends the "chain is a parameter" claim past the EVM family.
- **`contracts/src/`** — the Solidity home stack a Sui proxy bridges to.

Sui-based PolyMarkets — Overview

Justify prediction markets on Sui: the object-centric, Move-native implementation

Version: 1.0 (design phase)

Date: 2026-06-13

Status: Designed, not implemented. This is a forward-looking project separate from the EVM work. Where the EVM line (Arc, Base, Ethereum, Polygon, BSC) treats chain as a deployment parameter within the Solidity/EVM family, this project extends the "multi-chain protocol" thesis past EVM boundaries by reimplementing the same prediction-market product natively on **Sui** in **Move**.

---

1. Why Sui for prediction markets

Sui is a non-EVM, object-centric blockchain purpose-built for high-throughput, low-latency applications. Its design delivers five structural advantages for Justify's social prediction-market product:

Sui feature	Benefit for prediction markets
Object-centric model	Markets, positions, and AMM pools are first-class *objects* with explicit ownership (owned vs shared). A user's position is an owned object in their address; the AMM pool is a shared object multiple traders interact with. No global storage scans.
Parallel execution	Sui's transaction ordering is object-based, not account-based. Independent trades on different markets (or even different outcomes of the same market) can execute in parallel — no sequential bottleneck.
Move safety	Move's *resource types* enforce linear ownership (no double-spend or accidental duplication of positions), and the lack of reentrancy as a language primitive eliminates an entire class of AMM exploits.
Low fees	Sui's gas model is 10-100x cheaper than Ethereum mainnet; retail prediction-market trades (often $1-$50) become economically viable.
Sponsored transactions	A creator or the protocol can sponsor gas for new users, so a first-time trader pays zero fees — critical for social-consumer onboarding from Web2 audiences.
Native USDC on Sui	Circle's USDC is native on Sui (confirm on Sui docs for exact object/coin type). Markets denominated in USDC align with the same stablecoin-collateral model as the EVM stack.

1.1 The object-centric thesis

Traditional EVM contracts store state in account-scoped storage slots; reading a user's positions requires iterating events or maintaining off-chain indexes. Sui's object model inverts this: a position is an **owned object** directly held by the user's address. The chain itself is the index.

For Justify, this means:

- **Portfolio queries are object lookups.** "Show me Alice's positions" is an owned-object scan on Alice's address, not a log replay.
- **Parallel trades scale naturally.** Bob buying YES and Carol buying NO on the same market can execute concurrently if the Move module design allows per-outcome escrow.
- **Social objects are composable.** A market object can be *dynamically fields* of a creator's profile object, making the social graph and the trading graph one unified on-chain structure (though Justify's MVP keeps the social layer off-chain for cost/latency — Sui makes the on-chain social option viable later).

---

2. The object model mapping: EVM contracts → Sui objects

The EVM Justify stack (see `contracts/src/`) is a set of Solidity contracts, each with its own storage. Sui replaces "contract with storage" with "Move module + objects." The table below maps the EVM design to the Sui equivalent.

2.1 EVM → Sui contract equivalence

EVM component	Sui / Move equivalent	Key differences
MarketFactory.sol	Move module `market_factory` + a singleton shared object `FactoryRegistry`	EVM: factory is a contract with a `markets` mapping. Sui: registry is a shared object; `create_market()` is a public entry function that mints a new `Market` shared object.
PredictionMarket.sol	Shared object `Market` with fields: `id`, `question`, `outcome_labels`, `close_time`, `state`, `winning_outcome`, `creator`, `oracle_proof_url`	EVM: contract per market. Sui: object per market. State machine (Open/Closed/Resolved) is an enum in the object's `state` field.
MarketAMM.sol	Shared object `AMMPool` holding two `Balance<USDC>` reserves for YES/NO, paired 1:1 with a `Market` object	EVM: separate contract with its own storage. Sui: the pool is a shared object; `buy()` is a public entry function in the `amm` module that mutates the pool's reserves.
OutcomeToken.sol	`Coin<OUTCOME_YES>` and `Coin<OUTCOME_NO>` — Sui's native `Coin<T>` type with custom witness types per market outcome	EVM: ERC-1155 with tokenId encoding market+outcome. Sui: each market's YES/NO outcomes are distinct `Coin<T>` types (or a single `Coin<OUTCOME>` with dynamic fields per market).
OracleResolver.sol	Move module `oracle_resolver` + a shared `ResolverRegistry` object granting resolution capability to authorized resolvers	EVM: contract with role-based access. Sui: capability object pattern — a `ResolverCap` is an owned object held by the resolver; presenting it authorizes `resolve()` calls.
FeeTreasury.sol	Shared object `FeeTreasury` holding accumulated `Balance<USDC>` from AMM fees; withdrawal gated by a `TreasuryCap` capability	EVM: contract with role-guarded `withdraw()`. Sui: capability pattern + shared treasury object.
AccessControl.sol	Sui capability objects: `AdminCap`, `ResolverCap`, `TreasuryCap` — owned objects that grant authority	EVM: role-based access control via OpenZeppelin. Sui: capability-based; transferring the `AdminCap` object transfers admin authority — no role registry.
MockUSDC.sol	Native Circle USDC on Sui (or a testnet `Coin<MOCK_USDC>` for devnet)	EVM: ERC-20 contract. Sui: `Coin<T>` is a first-class primitive.

2.2 Object-centric vs account/storage model (comparison)

Aspect	EVM (Solidity)	Sui (Move)
State location	Contract storage (mappings, arrays, structs in contract slots)	Objects (owned, shared, or immutable) held by addresses or accessible globally by object ID
Market representation	`PredictionMarket` contract deployed once per market at a unique address; state in that contract's storage	`Market` shared object created once per market; state in the object's fields; object ID is the market identifier

Position representation	ERC-1155 token balance in `OutcomeToken` contract's mapping: `balances[user][tokenId]`	Owned `Coin<OUTCOME>` object held directly in the user's address; the chain tracks ownership
AMM pool reserves	`uint256[2] public reserves` in `MarketAMM.sol` storage	`Balance<USDC>` fields in the `AMMPool` shared object; mutated by `buy()` / `sell()` transactions
Access control	Role-based: `AccessControl` contract with `hasRole(role, account)` checks	Capability-based: presenting an owned `ResolverCap` object in a transaction proves authority; no global role registry
Reentrancy risk	Explicit guard (`ReentrancyGuard` modifier) required; external calls can reenter	Move does not allow reentrancy by design (no external calls during mutable borrows); resource types prevent duplication
Portfolio query	Off-chain: scan `Transfer` events or index into Postgres. On-chain: iterate all markets and check `balanceOf(user, id)`	On-chain: query owned objects at the user's address filtered by type `Coin<OUTCOME>` — the chain is the index
Parallel execution	Sequential per block (EVM is single-threaded); independent transactions still ordered by nonce	Parallel: if two trades touch disjoint objects (different markets or different outcome coins), Sui schedules them concurrently
Upgrade model	Proxy pattern (e.g., UUPS) or immutable contracts requiring redeployment	Move packages can be upgraded if published with upgrade capability; objects remain live, module logic is replaced

---

3. Product parity matrix: EVM stack → Sui Move modules

Every feature the EVM contracts provide must have a Sui equivalent. This table defines the functional mapping.

EVM contract function	Sui Move equivalent	Notes / differences
MarketFactory.createMarket()	`market_factory::create_market(registry: &mut FactoryRegistry, question, ...): Market`	EVM: returns marketId. Sui: returns (or transfers to caller) a new `Market` shared object. The factory registry records the object ID.
MarketAMM.seed()	`amm::seed_pool(pool: &mut AMMPool, seed_coin: Coin<USDC>, yes_amount, no_amount)`	EVM: transfers collateral from caller. Sui: caller passes a `Coin<USDC>` object; the function splits it and deposits into the pool's `Balance<USDC>` reserves.
MarketAMM.buy(outcomeIndex, collateralIn)	`amm::buy(pool: &mut AMMPool, market: &Market, collateral_in: Coin<USDC>, outcome_index, min_shares_out): Coin<OUTCOME>`	EVM: transfers ERC-20, mints ERC-1155. Sui: consumes input `Coin<USDC>`, mints and returns `Coin<OUTCOME_YES>` or `Coin<OUTCOME_NO>` to the caller.
MarketAMM.impliedProbabilityBps()	`amm::implied_probability_bps(pool: &AMMPool, outcome_index): u64`	Read-only; calculates price from the pool's

		reserves. Identical math (CPMM), same 0-10,000 basis-point range.
PredictionMarket.close()	`market::close(market: &mut Market, creator_cap: &CreatorCap)`	EVM: `onlyCreator` modifier checks `msg.sender`. Sui: capability pattern — only the holder of the `CreatorCap` can call `close()`.
PredictionMarket.resolve(winningOutcome)	`market::resolve(market: &mut Market, resolver_cap: &ResolverCap, winning_outcome: u8)`	EVM: role check. Sui: presenting `ResolverCap` proves authority. Updates `market.state` to `Resolved` and sets `market.winning_outcome`.
OutcomeToken.mint(to, id, amount)	`coin::mint<OUTCOME>(mint_cap: &mut TreasuryCap<OUTCOME>, amount): Coin<OUTCOME>`	Sui `Coin` standard uses `TreasuryCap<T>` for minting. The AMM module holds a `TreasuryCap<OUTCOME_YES>` and `TreasuryCap<OUTCOME_NO>` for each market.
OutcomeToken.burn(from, id, amount)	`coin::burn<OUTCOME>(treasury: &mut TreasuryCap<OUTCOME>, coin: Coin<OUTCOME>)`	Redemption: user transfers their `Coin<OUTCOME>` to a redemption function, which burns it and returns collateral from the pool.
OracleResolver.resolve(marketId, outcome)	`oracle_resolver::resolve(registry: &mut ResolverRegistry, market: &mut Market, resolver_cap: &ResolverCap, outcome, proof_url)`	Sui: same capability pattern. The resolver presents `ResolverCap`, the function mutates the `Market` object's state.
FeeTreasury.withdraw(to, amount)	`fee_treasury::withdraw(treasury: &mut FeeTreasury, cap: &TreasuryCap, amount): Coin<USDC>`	Sui: returns a `Coin<USDC>` to the caller. Caller can transfer it to any address.

3.1 Outcome token design: Coin vs dynamic fields

Two Move design options for outcome tokens:

1. **Per-market coin types.** Each market gets two distinct `Coin<T>` types: `Coin<MARKET_1_YES>`, `Coin<MARKET_1_NO>`, `Coin<MARKET_2_YES>`, etc. Pro: type safety — impossible to redeem Market 1 YES for Market 2's pool. Con: every market requires publishing a new coin type (or generating one-time witnesses dynamically).
2. **Single outcome coin type with dynamic fields.** `Coin<OUTCOME>` is a single type; each market's escrow distinguishes outcomes via dynamic fields on the pool object. Pro: one coin type for all markets. Con: more runtime checks; the redemption function must verify the coin belongs to the correct market.

Recommended approach: option 1 (per-market coin types) for maximum type safety, using Move's one-time witness pattern to generate unique `OUTCOME_YES` and `OUTCOME_NO` witness types at market creation. This mirrors the EVM design where ERC-

1155 token IDs encode market+outcome, but enforced at the type system level rather than runtime.

---

4. Hybrid on-chain / off-chain split: unchanged from EVM

The product's data architecture is identical on Sui:

Concern	Where it lives	Why
Money: collateral, positions, prices, settlement	**On-chain (Sui objects)**	Trust-minimized custody; auditable odds; the blockchain is the ledger.
Social: posts, follows, likes, comments, profiles	**Off-chain (Postgres, same backend as EVM deployment)**	Latency and cost — even Sui's low fees cannot match the sub-10ms, zero-cost expectation for social actions.
The link between them	Market object IDs referenced in posts; backend indexer mirrors Sui object state	Feed renders live odds by reading indexed state, not querying Sui RPC on every page load.

Implication: Justify on Sui reuses the **same Next.js backend, Postgres schema, and social graph** as the EVM deployment. Only the blockchain integration layer changes: wagmi/viem (EVM) → Sui TypeScript SDK; Solidity events → Sui transaction effects; ERC-20 approvals → `Coin` transfers.

The multi-chain product promise extends naturally: a user on Sui sees the same feed, the same profiles, and the same market cards as a user on Base — they are simply trading through different substrate layers that converge in the same Postgres social graph.

---

5. How Sui fits the multi-chain thesis: beyond EVM, not instead of EVM

The Justify whitepaper (§4) states:

> *The chain is a detail. The market is the product.*

Sui extends this thesis **beyond the EVM family**. Where Arc, Base, Ethereum, Polygon, and BSC are "chain as a config parameter" (same Solidity contracts, different RPC endpoints and collateral addresses), Sui is "chain as a substrate parameter" — the *product* is identical, the *implementation* is a port to a different VM and type system.

5.1 What stays the same

- **Market mechanics.** Binary YES/NO outcomes, CPMM pricing, collateral-backed shares, oracle resolution, the same 2% fee. A market on Sui behaves identically to a market on Base from the user's perspective.
- **The feed.** Markets from Sui, Base, and Ethereum all appear in the same chain-agnostic timeline, badged with their home network. A user follows a creator, not a chain.
- **The oracle-proof commitment.** Every market, on any chain, records an `oracle_proof_url` at creation — the public resolution source. The same off-chain oracle indexer (or Chainlink CRE-style automation) can resolve markets on Sui and EVM networks uniformly.
- **The social graph.** One Postgres database, one user table, one follow graph. A user's wallet address (EVM or Sui) maps to the same profile.

5.2 What changes: a reimplementation, not a config swap

Be honest: **deploying Justify to Sui is a reimplementation in Move, not a deployment script change.** Unlike the EVM stack, where adding Polygon meant updating environment variables and redeploying the same compiled bytecode, Sui requires:

1. **Rewriting contracts in Move.** The `contracts/src/*.sol` files have no Sui equivalent — they must be ported module by module to Move's resource-oriented type system.

2. **New client-side integration.** The frontend's wagmi hooks (`useContractWrite` , `useContractRead`) are EVM-specific. Sui requires the Sui TypeScript SDK, different wallet adapters (Sui Wallet, Suiet), and a different transaction-building flow (programmable transaction blocks instead of ABI-encoded calldata).
3. **Event indexing differences.** EVM emits logs; Sui produces transaction *effects* (a structured diff of object changes). The backend indexer must parse Sui events from the `Event` objects emitted by Move modules, not from EVM log topics.
4. **Different finality model.** Sui uses a DAG-based consensus (Narwhal + Bullshark); finality is certificate-based, not longest-chain. The settlement UX must reflect Sui's sub-second finality (versus Base's 2s or Ethereum's 12-15s).

5.3 Why do it, then?

Two strategic reasons:

1. **Reach.** Sui has a distinct user base and capital pool. Users with SUI and USDC on Sui cannot trade Justify markets on EVM chains without bridging — costly, slow, and a UX barrier. A native Sui deployment meets them where they are.
2. **Performance proof of concept.** Sui's parallel execution and object model are designed for high-throughput apps. Justify on Sui demonstrates the protocol can scale past EVM's sequential bottleneck — a technical and narrative win.

The product's multi-chain claim becomes: **we run on five EVM chains and Sui**, proving that "chain is a parameter" is not limited to the EVM monoculture.

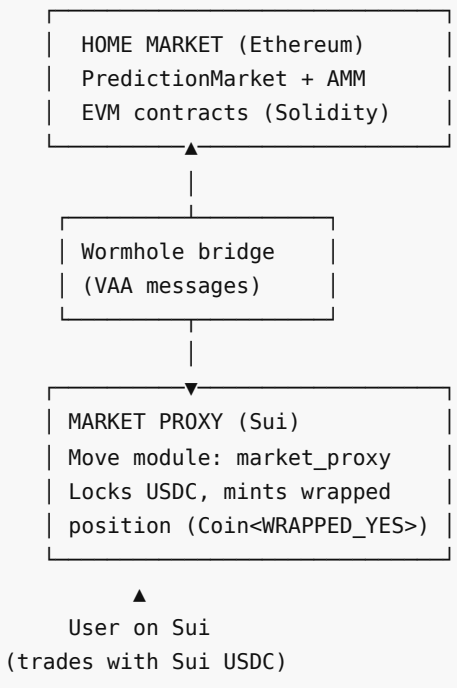
---

6. The Sui bridge: connecting Sui to the EVM home market

Once Justify has both EVM deployments (Arc, Base, Ethereum, Polygon, BSC) and a Sui deployment, the **Phase 3 bridge layer** (see `documentation_bridge/00-overview.md`) extends to Sui.

6.1 Sui as a proxy chain

A market created on Ethereum (home chain) can have a **Sui proxy**:



The Sui proxy is a Move module that:

1. Accepts a user's `Coin<USDC>` on Sui.
2. Locks it in escrow (a `Balance<USDC>` field in a shared `ProxyPool` object).
3. Emits a Sui event encoding the buy intent: `BuyIntent { market_id, user, outcome, amount, min_shares }` .
4. A **Wormhole relayer** (off-chain service) observes the Sui event, requests a signed VAA (Verifiable Action Approval) from Wormhole's guardian set, and submits it to the Ethereum home market's `WormholeAdapter` contract (EVM).

5. The home market's AMM executes the trade and emits a confirmation event.
6. The relayer carries the confirmation back to Sui as a Wormhole message; the Sui proxy verifies the VAA and mints a `Coin<WRAPPED_POSITION_YES>` to the user.

At resolution, the same flow in reverse: Ethereum resolves, Wormhole propagates the resolution to Sui, the proxy burns the wrapped position and returns the locked USDC.

6.2 Wormhole as the transport

Why Wormhole, not Chainlink CCIP? **Sui is non-EVM and has no Chainlink CCIP lane today.** Wormhole is the established Sui↔EVM general-message bridge with production volume. The architecture (documented in `documentation_sui/02-sui-bridge-overview.md` and `03-wormhole-adapter-and-interop.md`) keeps the same **Chainlink-CCIP-inspired layering** as the EVM bridge set:

- A chain-local router/adapter seam (`IBridgeAdapter` on EVM, `bridge_adapter` module on Sui).
- Message-carrying transport (Wormhole VAAs instead of CCIP DON attestations).
- Independent risk layer (exposure caps, circuit breaker — the RMN-inspired pattern).

The transport is pluggable: if Chainlink launches a Sui CCIP lane later, swapping Wormhole for CCIP is an adapter replacement, not a protocol redesign.

6.3 Sui as a home chain

A market can also be **created on Sui** and proxied to EVM chains. The home-market object lives on Sui; Base, Polygon, etc. deploy `MarketProxy` EVM contracts that forward intents via Wormhole to Sui. The Move AMM on Sui is the single source of truth.

Same model, inverted direction. The bridge architecture is symmetric.

---

7. Forward references — the Sui documentation set

This overview is the entry point. The full Sui project is detailed across three documents:

#	Document	What it covers
00	**00-sui-polymarkets-overview.md** (this)	Why Sui; object model vs EVM; product parity; the hybrid split; multi-chain thesis extension.
01	**01-move-market-contracts.md**	The Move package design: module structure, object definitions (`Market`, `AMMPool`, `Coin<OUTCOME>`), capability pattern, function signatures, CPMM math in Move.
02	**02-sui-bridge-overview.md**	The Sui bridge: Sui as proxy, Sui as home, Wormhole adapter (Move + EVM sides), VAA message flow, Chainlink-inspired layering on a non-EVM chain.

Supplementary:

- **03-wormhole-adapter-and-interop.md** (sibling to `documentation_bridge/`) — detailed Wormhole VAA encoding, guardian verification, Sui `Event` → EVM log and vice versa.
- **04-sui-bridge-security.md** — Wormhole Guardians vs Chainlink DON/RMN; exposure caps on Sui; circuit breaker via capability revocation.

---

8. Status and relation to the rest of the repo

Current phase: This is a **designed, not implemented** project. The EVM stack (see `contracts/src/`) is the delivered Phase 1 work; Sui is a separate forward-looking initiative.

Design freeze intent: These documents define the Sui Move package and bridge adapter interfaces so that, when the project begins, the integration contract between the Sui layer and the existing backend/frontend is frozen. The backend's market

indexer, the frontend's wallet connector, and the oracle resolver must support both EVM and Sui — this set defines what "support Sui" means.

Related documentation:

- **`documentation_justify_whitepaper/justify-whitepaper.md` §2-§4** — the product (social prediction markets), the hybrid split, the "chain is a parameter" thesis that Sui extends.
- **`documentation_bridge/00-overview.md`** — the EVM bridge (CCIP-based, home/proxy model). Sui mirrors this architecture with Wormhole as the transport.
- **`contracts/src/`** — the Solidity contracts Sui achieves parity with.
- **`documentation_polymarket_social/architecture-polymarket-platform-reference.md`** — how the real Polymarket does it (CLOB + CTF + UMA). Sui and EVM Justify both use on-chain CPMM, not CLOB — but the CTF conditional-token idea is the same (outcome coins redeem on resolution).

---

9. Open questions and next steps

Before implementation begins, confirm:

1. **Sui native USDC object type.** Circle has announced USDC on Sui; verify the exact `Coin<T>` type identifier and whether it is mainnet-live or testnet-only as of 2026-06-13. Update this doc with the canonical type.
2. **Outcome token type generation.** Decide between per-market coin types (unique one-time witness per market) vs a single `Coin<OUTCOME>` with dynamic fields. Lean toward per-market types for type safety unless Move package size limits force dynamic fields.
3. **Sponsored transaction flow.** Design which transactions the protocol sponsors (e.g., first trade per user) and how the gas-payer backend service integrates with the frontend's transaction builder.
4. **Wormhole guardian set vs Chainlink DON.** Understand Wormhole's 19-guardian threshold model and whether it meets the same security bar as Chainlink's decentralized oracle network. Document in `04-sui-bridge-security.md`.

Next: proceed to `01-move-market-contracts.md` for the Move package design — module layout, object structs, function signatures, capability pattern, and CPMM math translated from Solidity to Move.

End of overview. This document establishes *why* Sui and *what* changes from the EVM stack. The rest of the Sui documentation set defines *how* to build it.

01 — Move Market Contracts: Justify on Sui

Sui-native prediction markets: object-centric, resource-safe, bridgeable

This document specifies the **Move package design** for Justify prediction markets on Sui. The package is a faithful re-expression of the EVM stack (`contracts/src/`) in Sui's object model: factory-deployed markets, constant-product AMM pricing, outcome positions as Coin types, capability-based access control, and explicit event emissions for bridge and indexer consumption. The design prioritizes **object-centric safety** (no reentrancy, resource conservation enforced by type system) and **bridge interop** (entry points the Wormhole adapter calls, events the MarketProxy consumes).

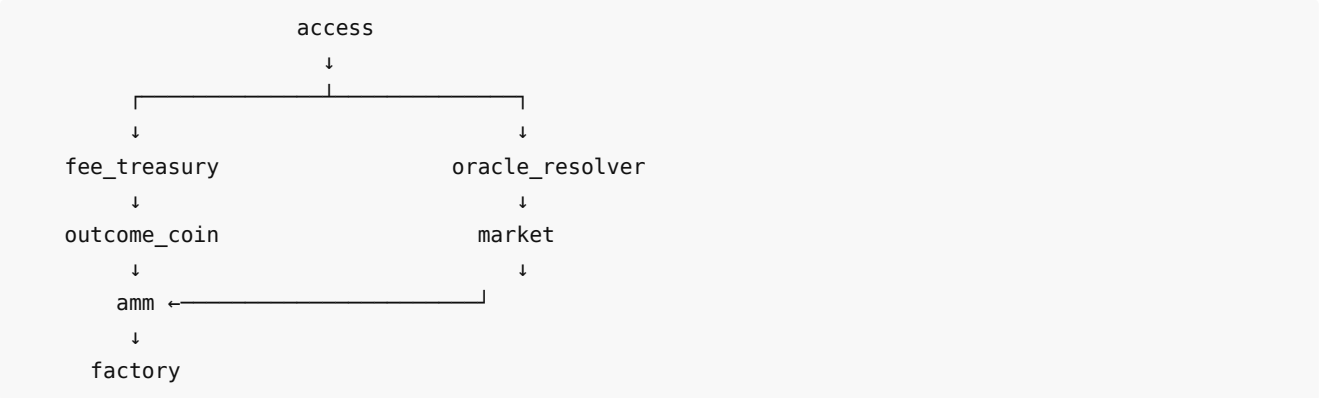
---

1. Package layout

The Move package is named **`justify_markets`** and organized into seven modules:

```
justify_markets/
├── sources/
│   ├── factory.move          // MarketRegistry + createMarket (analog: MarketFactory.sol)
│   ├── market.move           // Market shared object (analog: PredictionMarket.sol)
│   ├── amm.move              // CPMM pool, buy/sell (analog: MarketAMM.sol)
│   ├── outcome_coin.move     // YES/NO outcome coins (analog: OutcomeToken.sol ERC-1155)
│   ├── oracle_resolver.move  // Resolution logic (analog: OracleResolver.sol)
│   ├── fee_treasury.move     // Fee collection (analog: FeeTreasury.sol)
│   └── access.move           // Capability objects for roles (analog: AccessControl.sol)
├── Move.toml
└── tests/
    └── market_tests.move
```

Module dependency tree:



- `access` defines capability objects (`AdminCap` , `ResolverCap` , `FactoryCap`).
- `market` holds the market state machine (Open → Closed → Resolved).
- `outcome_coin` defines the `YES` and `NO` fungible outcome types.
- `amm` implements constant-product pricing and mints/burns outcome coins.
- `factory` deploys new markets and registers them in the shared `MarketRegistry` .
- `oracle_resolver` records resolution decisions and updates market state.
- `fee_treasury` collects trading fees (2% per trade, matching EVM 200 BPS).

---

2. Core objects and structs

2.1 `Market` — shared object

The `Market` is a **shared object** (`has key, has store`) representing one prediction market. It is the Sui analog to `PredictionMarket.sol` .

```
module justify_markets::market {
    use sui::object::{Self, UID};
    use sui::coin::{Coin, Balance};
    use sui::balance::{Self, Balance};
    use sui::tx_context::{Self, TxContext};
    use sui::transfer;

    /// State machine: Open → Closed → Resolved
    struct MarketState has copy, drop, store {
        open: bool,
        closed: bool,
        resolved: bool,
    }

    /// The Market shared object (one per market).
    struct Market has key, store {
        id: UID,
        market_id: u64,              // Globally unique ID (factory counter)
        question: vector<u8>,        // Market question (UTF-8 encoded string)
        outcome_labels: vector<vector<u8>>, // ["YES", "NO"] or ["Barcelona", "Real Madrid"]
        close_time: u64,             // Unix timestamp; 0 = no fixed close
        state: MarketState,
        winning_outcome: u8,         // 0 or 1; set on resolution
        creator: address,
        oracle_proof_url: vector<u8>,// URL to oracle reference (UTF-8)

        // AMM pool state (embedded in Market for simplicity)
        collateral: Balance<USDC>,   // Locked collateral backing outcome shares
        reserves: vector<u64>,       // [YES reserve, NO reserve] in basis units
        fee_bps: u64,                // Fee in basis points (200 = 2%)

        // Outcome supply tracking
        yes_supply: u64,
        no_supply: u64,
    }

    /// Event: Market created
    struct MarketCreated has copy, drop {
        market_id: u64,
        question: vector<u8>,
        creator: address,
    }

    /// Event: Market closed (trading stops)
    struct MarketClosed has copy, drop {
        market_id: u64,
    }

    /// Event: Market resolved
    struct MarketResolved has copy, drop {
        market_id: u64,
        winning_outcome: u8,
        oracle_proof_url: vector<u8>,
```

```
    }
}
```

Design notes:

- **Shared object:** `Market` is shared (not owned) so any user can reference it in transactions without explicit transfers. The AMM pool state (`reserves` , `collateral`) lives directly in the `Market` struct, eliminating the separate `MarketAMM` contract from the EVM design — Sui's object model makes embedding more natural.
- **State machine:** `MarketState` mirrors `PredictionMarket.sol` 's `Open | Closed | Resolved` enum. The `state` field is checked in entry functions to gate trading after close or resolution.
- **Collateral as `Balance<USDC>` :** The locked collateral is a `Balance<USDC>` (Sui's native type for coin reserves). This is a **resource type** — the Move type system guarantees it cannot be double-spent or lost.
- **Outcome supplies:** `yes_supply` and `no_supply` track total minted outcome coins for accounting (analog: ERC-1155 `totalSupply` per token ID).

2.2 `MarketRegistry` — shared factory state

The `MarketRegistry` is a **shared object** tracking all created markets (analog: `MarketFactory.sol` 's `markets` mapping).

```
module justify_markets::factory {
    use sui::object::{Self, UID};
    use sui::table::{Self, Table};
    use sui::tx_context::{Self, TxContext};

    struct MarketRegistry has key {
        id: UID,
        next_market_id: u64,             // Auto-increment counter
        markets: Table<u64, address>,    // market_id → Market object address
    }

    /// Event: Registry initialized
    struct RegistryCreated has copy, drop {
        registry_id: address,
    }
}
```

Design notes:

- **`Table<u64, address>` :** Sui's `Table` is an off-chain-indexable map (on-chain key-value store with lazy loading). It maps `market_id` to the `Market` object's address (Sui object IDs are addresses).
- **Singleton pattern:** One `MarketRegistry` is created at package publish time and shared. All markets register themselves in this table.

2.3 Outcome coins — `Coin<YES>` and `Coin<NO>`

Sui's idiomatic way to represent fungible assets is the **`Coin<T>` type** with a one-time witness (OTW). Each outcome is a distinct coin type.

```
module justify_markets::outcome_coin {
    use sui::coin::{Self, Coin, TreasuryCap};
    use sui::transfer;
    use sui::tx_context::{Self, TxContext};

    /// One-time witness for YES outcome
    struct YES has drop {}

    /// One-time witness for NO outcome
```

```
struct NO has drop {}

/// Treasury capabilities (held by AMM module, never transferred)
struct YesTreasury has key {
    id: UID,
    cap: TreasuryCap<YES>,
}

struct NoTreasury has key {
    id: UID,
    cap: TreasuryCap<NO>,
}

/// Initialize outcome coin types (called once at package publish)
fun init(ctx: &mut TxContext) {
    // Create YES coin type with 6 decimals (matching USDC)
    let (yes_treasury_cap, yes_metadata) = coin::create_currency(
        YES {},
        6,
        b"YES",
        b"Justify YES Outcome",
        b"Conditional outcome token for YES",
        option::none(),
        ctx
    );

    // Create NO coin type
    let (no_treasury_cap, no_metadata) = coin::create_currency(
        NO {},
        6,
        b"NO",
        b"Justify NO Outcome",
        b"Conditional outcome token for NO",
        option::none(),
        ctx
    );

    // Share metadata objects for wallets/explorers
    transfer::public_freeze_object(yes_metadata);
    transfer::public_freeze_object(no_metadata);

    // Store treasury caps in shared objects (AMM module will access)
    transfer::share_object(YesTreasury {
        id: object::new(ctx),
        cap: yes_treasury_cap,
    });

    transfer::share_object(NoTreasury {
        id: object::new(ctx),
        cap: no_treasury_cap,
    });
}

/// Mint YES coins (called by AMM on buy)
public fun mint_yes(
    treasury: &mut YesTreasury,
```

```
        amount: u64,
        ctx: &mut TxContext
    ): Coin<YES> {
        coin::mint(&mut treasury.cap, amount, ctx)
    }

    /// Mint NO coins
    public fun mint_no(
        treasury: &mut NoTreasury,
        amount: u64,
        ctx: &mut TxContext
    ): Coin<NO> {
        coin::mint(&mut treasury.cap, amount, ctx)
    }

    /// Burn YES coins (on redemption after resolution)
    public fun burn_yes(
        treasury: &mut YesTreasury,
        coin: Coin<YES>
    ) {
        coin::burn(&mut treasury.cap, coin);
    }

    /// Burn NO coins
    public fun burn_no(
        treasury: &mut NoTreasury,
        coin: Coin<NO>
    ) {
        coin::burn(&mut treasury.cap, coin);
    }
}
```

Design rationale:

- **Why `Coin<YES>` / `Coin<NO>` instead of `OutcomePosition` owned object?** Sui's `Coin` type is the standard for fungible assets. It integrates natively with wallets (Sui Wallet, Martian), DEX protocols (Cetus, Turbos), and indexers. An `OutcomePosition` owned object would require custom wallet support and transfer logic.
- **One-time witness (OTW):** The `YES` and `NO` structs with `has drop` ability are OTWs — they can be constructed exactly once (at module init) to create unique coin types. This prevents anyone else from minting counterfeit YES/NO coins.
- **Treasury capabilities:** The `TreasuryCap<YES>` and `TreasuryCap<NO>` are held in **shared objects** (`YesTreasury`, `NoTreasury`) that the AMM module accesses via `&mut` references. This is the Move analog to granting `MINTER_ROLE` to `MarketAMM` in the EVM stack.

Trade-off acknowledged: `Coin<YES>` is a **global type** (one YES type for ALL markets). To distinguish YES shares from market #1 vs market #2, the system must rely on:

1. **AMM invariant:** The AMM only mints YES for a specific market and burns it on redemption from that same market.
2. **Indexer mapping:** The off-chain indexer tracks which `Coin<YES>` issuances correspond to which market via event emissions.

An alternative design is **per-market coin types** (e.g. `Coin<Market42_YES>` via dynamic OTW generation). This is more complex but type-safe. For the MVP, the global YES/NO types with off-chain indexing match the EVM ERC-1155 approach where token IDs encode market + outcome.

2.4 AMM pool state (embedded in `Market`)

In the EVM stack, `MarketAMM.sol` is a separate contract. In Sui, the AMM state lives **inside the `Market` shared object** to minimize object proliferation. The key fields:

```
// Inside Market struct
collateral: Balance<USDC>,   // Locked collateral (6-decimal units)
reserves: vector<u64>,       // [YES reserve, NO reserve]
fee_bps: u64,                // 200 basis points (2%)
yes_supply: u64,             // Total YES coins minted for this market
no_supply: u64,              // Total NO coins minted for this market
```

The AMM module (`justify_markets::amm`) contains the entry functions that mutate these fields.

2.5 Capability objects for access control

Move's **capability pattern** replaces Solidity's role-based `AccessControl.sol` . A capability is an owned object that grants its holder the right to perform privileged operations.

```
module justify_markets::access {
    use sui::object::{Self, UID};
    use sui::transfer;
    use sui::tx_context::{Self, TxContext};

    /// Admin capability (held by deployer/protocol owner)
    struct AdminCap has key, store {
        id: UID,
    }

    /// Resolver capability (held by oracle service)
    struct ResolverCap has key, store {
        id: UID,
    }

    /// Factory capability (held by factory module)
    struct FactoryCap has key, store {
        id: UID,
    }

    /// Initialize capabilities (called once at package publish)
    fun init(ctx: &mut TxContext) {
        // Mint and transfer AdminCap to deployer
        transfer::transfer(AdminCap {
            id: object::new(ctx),
        }, tx_context::sender(ctx));

        // Mint ResolverCap and transfer to deployer (can be delegated)
        transfer::transfer(ResolverCap {
            id: object::new(ctx),
        }, tx_context::sender(ctx));

        // Mint FactoryCap and share it (factory module holds &mut reference)
        transfer::share_object(FactoryCap {
            id: object::new(ctx),
        });
    }

    /// Transfer AdminCap to a new admin (only holder can call)
```

```
    public entry fun transfer_admin_cap(
        cap: AdminCap,
        new_admin: address
    ) {
        transfer::transfer(cap, new_admin);
    }

    /// Transfer ResolverCap to oracle service
    public entry fun transfer_resolver_cap(
        cap: ResolverCap,
        oracle: address
    ) {
        transfer::transfer(cap, oracle);
    }
}
```

Design notes:

- **No role mappings:** Unlike Solidity's `mapping(bytes32 => mapping(address => bool))`, Move capabilities are **objects**. To check if a caller has admin rights, the function signature requires `admin_cap: &AdminCap` as a parameter — no on-chain lookup.
- **Transfer semantics:** Capabilities with `has store` can be transferred peer-to-peer via `transfer::transfer`. This is the Move equivalent of `grantRole / revokeRole`.
- **FactoryCap as shared object:** The `FactoryCap` is shared (not owned) so the factory module can reference it in `create_market` calls without explicit transfers. This is an ergonomic trade-off — a fully permission-less factory would not need a capability at all, but the cap pattern allows future gating (e.g. allowlists for market creation).

---

3. Key entry functions

Entry functions are `public entry fun` — callable from transactions, analogous to Solidity's `external` functions.

3.1 `create_market` (factory module)

```
module justify_markets::factory {
    use justify_markets::market::{Self, Market};
    use justify_markets::access::FactoryCap;
    use sui::tx_context::{Self, TxContext};

    /// Create a new prediction market.
    /// Requires FactoryCap (gating future: e.g., allowlist checks).
    public entry fun create_market(
        registry: &mut MarketRegistry,
        _factory_cap: &FactoryCap,  // Proves caller is authorized
        question: vector<u8>,
        outcome_labels: vector<vector<u8>>,
        close_time: u64,
        oracle_proof_url: vector<u8>,
        ctx: &mut TxContext
    ) {
        let market_id = registry.next_market_id;
        registry.next_market_id = market_id + 1;

        let market = market::new(
            market_id,
            question,
```

```
            outcome_labels,
            close_time,
            tx_context::sender(ctx),
            oracle_proof_url,
            ctx
        );

        let market_address = object::id_address(&market);
        table::add(&mut registry.markets, market_id, market_address);

        // Share the Market object (makes it accessible to all users)
        transfer::share_object(market);

        // Emit event
        event::emit(market::MarketCreated {
            market_id,
            question,
            creator: tx_context::sender(ctx),
        });
    }
}
```

Analog: `MarketFactory.createMarket()` in Solidity. The key difference: no separate `MarketAMM` contract deployment — the AMM state is embedded in the `Market` object.

3.2 `buy` (AMM module)

```
module justify_markets::amm {
    use justify_markets::market::{Self, Market};
    use justify_markets::outcome_coin::{Self, YES, NO, YesTreasury, NoTreasury};
    use sui::coin::{Self, Coin};
    use sui::tx_context::{Self, TxContext};
    use sui::transfer;

    /// Buy outcome shares via constant-product AMM.
    /// Mirrors MarketAMM.sol::buy logic.
    public entry fun buy(
        market: &mut Market,
        yes_treasury: &mut YesTreasury,
        no_treasury: &mut NoTreasury,
        collateral_in: Coin<USDC>,
        outcome_index: u8,  // 0 = YES, 1 = NO
        min_shares_out: u64,
        ctx: &mut TxContext
    ) {
        // 1. Check market state (must be Open)
        assert!(market.state.open && !market.state.closed, EMarketClosed);

        // 2. Deduct fee (200 bps = 2%)
        let collateral_amount = coin::value(&collateral_in);
        let fee = (collateral_amount * market.fee_bps) / 10_000;
        let net_collateral = collateral_amount - fee;

        // 3. CPMM swap (same logic as MarketAMM.sol)
        let reserves = &market.reserves;
```

```
        let other_index = 1 - outcome_index;
        let k = (*vector::borrow(reserves, 0) * *vector::borrow(reserves, 1));

        let new_other_reserve = *vector::borrow(reserves, other_index as u64) + net_collateral;
        let new_chosen_reserve = k / new_other_reserve;
        let shares_out = *vector::borrow(reserves, outcome_index as u64) - new_chosen_reserve;

        // 4. Slippage check
        assert!(shares_out >= min_shares_out, ESlippageExceeded);

        // 5. Update reserves
        *vector::borrow_mut(&mut market.reserves, outcome_index as u64) = new_chosen_reserve;
        *vector::borrow_mut(&mut market.reserves, other_index as u64) = new_other_reserve;

        // 6. Transfer fee to treasury (simplified: burn for MVP)
        let fee_coin = coin::split(&mut collateral_in, fee, ctx);
        transfer::public_transfer(fee_coin, fee_treasury_address());

        // 7. Lock remaining collateral in market
        balance::join(&mut market.collateral, coin::into_balance(collateral_in));

        // 8. Mint outcome coins to buyer
        let outcome_coin = if (outcome_index == 0) {
            market.yes_supply = market.yes_supply + shares_out;
            coin::from_balance(outcome_coin::mint_yes(yes_treasury, shares_out, ctx), ctx)
        } else {
            market.no_supply = market.no_supply + shares_out;
            coin::from_balance(outcome_coin::mint_no(no_treasury, shares_out, ctx), ctx)
        };

        transfer::public_transfer(outcome_coin, tx_context::sender(ctx));

        // 9. Emit event
        event::emit(TradeExecuted {
            market_id: market.market_id,
            trader: tx_context::sender(ctx),
            outcome_index,
            collateral_in: collateral_amount,
            shares_out,
            fee,
        });
    }

    /// Event: Trade executed
    struct TradeExecuted has copy, drop {
        market_id: u64,
        trader: address,
        outcome_index: u8,
        collateral_in: u64,
        shares_out: u64,
        fee: u64,
    }

    const EMarketClosed: u64 = 1;
    const ESlippageExceeded: u64 = 2;
}
```

Analog: `MarketAMM.buy()` in Solidity. The Move version uses Sui's `Balance` and `Coin` APIs instead of ERC-20 `safeTransferFrom`. The CPMM math (`k = reserves[0] * reserves[1]`) is identical to the Solidity implementation.

3.3 `sell` (AMM module)

```
public entry fun sell(
    market: &mut Market,
    yes_treasury: &mut YesTreasury,
    no_treasury: &mut NoTreasury,
    outcome_coin: Coin<YES>, // or Coin<NO>, depending on outcome_index
    outcome_index: u8,
    min_collateral_out: u64,
    ctx: &mut TxContext
) {
    // 1. Check market state
    assert!(market.state.open && !market.state.closed, EMarketClosed);

    let shares_in = coin::value(&outcome_coin);

    // 2. CPMM reverse swap (shares → collateral)
    // (Logic mirrors buy in reverse; omitted for brevity)
    let collateral_out = /* compute via reserves */;
    assert!(collateral_out >= min_collateral_out, ESlippageExceeded);

    // 3. Burn outcome coins
    if (outcome_index == 0) {
        outcome_coin::burn_yes(yes_treasury, outcome_coin);
        market.yes_supply = market.yes_supply - shares_in;
    } else {
        outcome_coin::burn_no(no_treasury, outcome_coin);
        market.no_supply = market.no_supply - shares_in;
    };

    // 4. Release collateral to seller
    let payout = coin::take(&mut market.collateral, collateral_out, ctx);
    transfer::public_transfer(payout, tx_context::sender(ctx));

    // Emit event...
}
```

Note: Sell flow is **deferred to post-MVP** in the EVM stack (functional-requirements.md §15 item 4). The Move design includes it for completeness, but it may not be exercised in the initial Sui deployment.

3.4 `resolve` (oracle resolver module)

```
module justify_markets::oracle_resolver {
    use justify_markets::market::{Self, Market};
    use justify_markets::access::ResolverCap;
    use sui::tx_context::{Self, TxContext};

    /// Resolve a market (close trading, set winning outcome).
    /// Requires ResolverCap (oracle authority).
    public entry fun resolve(
        market: &mut Market,
        _resolver_cap: &ResolverCap,  // Proves caller is authorized oracle
```

```
        winning_outcome: u8,
        ctx: &mut TxContext
    ) {
        // 1. Check market is Closed (must close before resolving)
        assert!(market.state.closed && !market.state.resolved, EInvalidState);

        // 2. Validate outcome index (0 or 1)
        assert!(winning_outcome <= 1, EInvalidOutcome);

        // 3. Update market state
        market.state.resolved = true;
        market.winning_outcome = winning_outcome;

        // 4. Emit resolution event
        event::emit(market::MarketResolved {
            market_id: market.market_id,
            winning_outcome,
            oracle_proof_url: market.oracle_proof_url,
        });
    }

    const EInvalidState: u64 = 10;
    const EInvalidOutcome: u64 = 11;
}
```

Analog: `OracleResolver.resolve()` in Solidity. The Move version gates the call with `ResolverCap` instead of checking `acl.hasRole(RESOLVER_ROLE, msg.sender)`.

3.5 `redeem` (AMM module)

```
public entry fun redeem(
    market: &mut Market,
    yes_treasury: &mut YesTreasury,
    no_treasury: &mut NoTreasury,
    outcome_coin: Coin<YES>, // or Coin<NO>
    outcome_index: u8,
    ctx: &mut TxContext
) {
    // 1. Check market is Resolved
    assert!(market.state.resolved, ENotResolved);

    let shares = coin::value(&outcome_coin);

    // 2. Burn outcome coins
    if (outcome_index == 0) {
        outcome_coin::burn_yes(yes_treasury, outcome_coin);
        market.yes_supply = market.yes_supply - shares;
    } else {
        outcome_coin::burn_no(no_treasury, outcome_coin);
        market.no_supply = market.no_supply - shares;
    };

    // 3. Payout: 1:1 if winning, 0 if losing
    let payout_amount = if (outcome_index == market.winning_outcome) {
        shares  // Each winning share redeems for 1 collateral unit
```

```
    } else {
        0
    };

    if (payout_amount > 0) {
        let payout = coin::take(&mut market.collateral, payout_amount, ctx);
        transfer::public_transfer(payout, tx_context::sender(ctx));
    };

    // Emit event
    event::emit(Redeemed {
        market_id: market.market_id,
        trader: tx_context::sender(ctx),
        outcome_index,
        shares_redeemed: shares,
        payout: payout_amount,
    });
}

struct Redeemed has copy, drop {
    market_id: u64,
    trader: address,
    outcome_index: u8,
    shares_redeemed: u64,
    payout: u64,
}

const ENotResolved: u64 = 3;
```

Analog: The redemption logic from `MarketAMM` / `OutcomeToken` after resolution. In Sui, the user passes their `Coin<YES>` or `Coin<NO>` object to the `redeem` function, which burns it and releases collateral.

---

4. AMM math — constant product and fee routing

The AMM uses the **same constant-product formula** as `MarketAMM.sol`:

4.1 Invariant

```
k = reserves[YES] * reserves[NO]
```

On each buy:

1. **Fee deduction:** `fee = collateral_in * fee_bps / 10_000` (200 bps = 2%)
2. **Net collateral:** `net = collateral_in - fee`
3. **Reserves update:**
   - `reserves[other] += net`
   - `reserves[chosen] = k / reserves[other]` (preserves `k`)
4. **Shares out:** `shares_out = old_reserves[chosen] - new_reserves[chosen]`

4.2 Implied probability

The price of YES is:

```
price_yes = reserves[NO] / (reserves[YES] + reserves[NO])
```

Displayed in the UI as **cents** (basis points * 100). Example: if `reserves[YES] = 600` and `reserves[NO] = 400`, then `price_yes = 400 / 1000 = 0.40 = 40¢`.

4.3 Fee routing

Fees are transferred to a **fee treasury address** (held by `AdminCap` holder). In the MVP, the fee is sent immediately on each trade:

```
let fee_coin = coin::split(&mut collateral_in, fee, ctx);
transfer::public_transfer(fee_coin, FEE_TREASURY_ADDRESS);
```

Known gap: The EVM stack has a `FeeTreasury` contract with withdrawal gating. The Move version simplifies this to a direct transfer. A production version would use a shared `FeeTreasury` object with `AdminCap`-gated withdrawals, mirroring the Solidity design.

4.4 Collateral conservation invariant

At all times:

```
market.collateral.value()
  == (market.yes_supply + market.no_supply) - redeemed_shares
```

This is enforced by Move's resource type system: the `Balance<USDC>` inside `market.collateral` cannot be created or destroyed except via `coin::take` (on redemption) and `coin::join` (on buy). No reentrancy or external call can violate this.

---

5. Move-specific safety wins

5.1 Resource types prevent double-spend

`Balance<USDC>` and `Coin<YES>` are **resource types** (the Move VM tracks them via linear logic). A `Coin<YES>` object cannot be:

- **Copied** (no `has copy` ability) — prevents double-spend of outcome shares.
- **Dropped** (no implicit `has drop`) — the compiler errors if a coin is not consumed (transferred, burned, or merged). This eliminates accidental loss of value.

Contrast with Solidity: ERC-1155 balances are `uint256` in a mapping — a malicious or buggy contract can overwrite them. In Move, the type system enforces conservation.

5.2 No reentrancy

Sui does not have reentrant calls (no `call` opcode). External contract calls are sequenced via **object references** (`&mut Market`) — the VM prevents two transactions from mutating the same shared object concurrently.

Result: The `buy` function in Solidity requires `nonReentrant` modifier from OpenZeppelin. The Move version needs no such guard — reentrancy is architecturally impossible.

5.3 Explicit object ownership

Every object has an **owner**:

- **Owned objects** (e.g., `AdminCap`) can only be used by their owner's transactions.
- **Shared objects** (e.g., `Market`, `MarketRegistry`) can be accessed by anyone but are locked during mutation (`&mut` reference).

Result: Access control is **type-enforced**. To call `resolve`, you must pass `&ResolverCap` — the Move VM checks at transaction-signing time that you own a `ResolverCap` object. No runtime `require(acl.hasRole(...))` needed.

5.4 Events via `event::emit`

Sui events are emitted via `sui::event::emit` and indexed off-chain by the Sui indexer (GraphQL or JSON-RPC subscription). The events are **type-safe** (structs with `copy + drop` abilities) and guaranteed to match the emitted data.

```
event::emit(TradeExecuted {
    market_id: market.market_id,
    trader: tx_context::sender(ctx),
    outcome_index,
    collateral_in,
    shares_out,
    fee,
});
```

Analog: Solidity `emit Trade(...)` events. The key difference: Sui events include the transaction digest and timestamp in the indexer's event stream, simplifying off-chain reconciliation (no need to parse block logs).

---

6. What the bridge needs from these contracts

The Wormhole adapter (see documentation_sui/03-wormhole-adapter-and-interop.md) requires these entry points and events:

6.1 Entry points for cross-chain operations

These functions are called by the **Wormhole relayer** or **MarketProxy** via Sui programmable transactions:

Entry point	Purpose	Caller
`amm::buy_on_behalf`	Execute a buy for a remote trader (EVM user via MarketProxy)	Wormhole adapter contract on Sui
`amm::credit_position`	Mint outcome coins to a remote user's Sui address (after EVM buy settled)	Wormhole adapter
`market::mark_resolved`	Update market state when resolution is bridged from EVM home chain	Wormhole adapter + ResolverCap

Not yet designed: `buy_on_behalf` and `credit_position` are bridge-specific entry points. The MVP may skip them if the bridge design uses **wrapped receipts** (ERC-20 on EVM, no native Sui outcome coins until redemption). See documentation_bridge/03-marketproxy-and-home-market.md for the wrapping model.

6.2 Events for the bridge and off-chain indexer

The bridge consumes these events to trigger cross-chain actions:

Event	Emitted by	Consumed by
`TradeExecuted`	`amm::buy` / `amm::sell`	Bridge adapter (to send FillConfirmation to EVM proxy)
`MarketResolved`	`oracle_resolver::resolve`	Bridge adapter (to broadcast resolution to all EVM proxies)
`Redeemed`	`amm::redeem`	Indexer (portfolio service updates unrealized P&L)
`MarketCreated`	`factory::create_market`	Indexer (market registry for UI)

Event payloads must match the bridge's message schemas. Example: `TradeExecuted` includes:

- `market_id` (maps to EVM `homeMarketId`)
- `trader` (maps to EVM proxy user address)

- `outcome_index` (0/1 for YES/NO)
- `shares_out` (minted shares, used by proxy to mint wrapped receipt)

6.3 Sui-to-EVM price feed

The AMM's `implied_probability_bps` view function (added to `amm` module) returns the current YES price in basis points:

```
public fun implied_probability_bps(market: &Market, outcome_index: u8): u64 {
    let total = *vector::borrow(&market.reserves, 0) + *vector::borrow(&market.reserves, 1);
    if (total == 0) { return 0 };
    let other_index = 1 - outcome_index;
    (*vector::borrow(&market.reserves, other_index as u64) * 10_000) / total
}
```

The EVM `MarketProxy` can call this (via Wormhole attestation) to display the home-chain price before a remote user submits a buy intent.

---

7. Comparison with EVM stack

Aspect	EVM (Solidity)	Sui (Move)
Market state	`PredictionMarket` contract	`Market` shared object
AMM pool	Separate `MarketAMM` contract	Embedded in `Market` struct
Outcome shares	ERC-1155 token IDs (`marketId << 1	outcome`)
Access control	`AccessControl` role mappings	Capability objects (`AdminCap`, `ResolverCap`)
Fee treasury	`FeeTreasury` contract with withdrawal gating	Direct transfer to admin address (MVP simplification)
Reentrancy	`nonReentrant` modifier (OpenZeppelin)	Architecturally impossible (no reentrant calls)
Collateral conservation	Manual accounting + SafeERC20	Type system enforced (`Balance<USDC>` is a resource)
Event indexing	Ethereum logs via `eth_getLogs`	Sui indexer GraphQL / JSON-RPC subscriptions

Design fidelity: The Move package faithfully reproduces the EVM stack's **behavior** (CPMM pricing, outcome shares, resolution state machine) while adapting to Sui's **object model** (shared objects, capabilities, resource types). Function names (e.g., `buy`, `resolve`, `redeem`) match the Solidity originals where semantics align.

---

8. Open questions and confirmations needed

Question	Status	Notes
Sui framework version	Unconfirmed	This design assumes Sui Move framework version 2024 or later (with `sui::coin`, `sui::balance`, `sui::event` modules). Confirm against deployed mainnet version.
`Coin<YES>` global scope	Design decision	Global YES/NO types (one per package) vs per-market coin types (e.g., `Coin<Market42_YES>` via dynamic witness). Global is simpler; per-market is type-safer. MVP uses global.
Fee treasury	Simplified	EVM has `FeeTreasury` contract with `TREASURY_ROLE` withdrawal gating. Move version transfers fees directly to admin address on each trade. Production may add a shared `FeeTreasury` object.

Bridge entry points	Deferred	`buy_on_behalf` and `credit_position` are forward-declared for Wormhole adapter. Design depends on wrapped-receipt vs native-coin bridge model (see doc 02).
USDC on Sui	External dependency	The package assumes `USDC` is a `Coin<USDC>` type deployed on Sui (likely via Wormhole-wrapped USDC or Circle native USDC when available). Address TBD.

---

9. Forward links

- **00-sui-polymarkets-overview.md** — Why Sui; the object-centric prediction-market model.
- **02-sui-bridge-overview.md** — The Sui bridge: home/proxy model on Sui, Chainlink-CCIP-inspired layering, Wormhole transport.
- **03-wormhole-adapter-and-interop.md** — The Wormhole adapter (Move + EVM sides), VAAs, Sui↔EVM message flow.
- **04-sui-bridge-security.md** — Security & risk; Wormhole Guardians vs Chainlink RMN; exposure caps, circuit breaker.
- **contracts/src/** — The EVM Solidity contracts this Move package mirrors in behavior.

END OF DOCUMENT

This Move package design ensures that Justify markets on Sui achieve **object-centric safety** (resource types, no reentrancy, capability-enforced access control), **functional parity** with the EVM stack (CPMM pricing, outcome shares, resolution state machine), and **bridge readiness** (entry points and events the Wormhole adapter consumes). The package is a faithful re-expression of the EVM contracts in Sui's programming model, designed for a Sui-native deployment that bridges to the EVM home/proxy network.

Sui Bridge Overview — connecting Sui markets to the EVM network

Document: 02-sui-bridge-overview.md
Status: Design (separate project, forward-looking)
Related: 00-sui-polymarkets-overview.md | 01-move-market-contracts.md | 03-wormhole-adapter-and-interop.md

---

1. Goal — unified liquidity across Sui and EVM

The Sui bridge solves the same problem as the EVM bridge set (see `documentation_bridge/00-overview.md`): **liquidity fragmentation by chain for the same event**. Where the EVM bridge unifies Arc, Base, Ethereum, Polygon, and BSC into one logical market per event, the Sui bridge extends that unification **across the EVM/non-EVM boundary** — connecting Sui markets to the existing EVM home/proxy network.

Two integration directions:

1.1 Sui as a PROXY chain (recommended initial path)

- **Home market:** lives on an EVM chain (e.g., Arc, Base, Ethereum) with its `PredictionMarket`, `MarketAMM`, collateral pool, and `OracleResolver`.
- **Sui proxy:** a lightweight Move package on Sui that accepts Sui USDC, locks it, emits a bridge message (buy/sell intent) to the EVM home market via Wormhole, receives confirmation, and mints a **wrapped position receipt** on Sui representing the outcome shares held in the EVM AMM pool.
- **Result:** Sui users trade on markets created on EVM chains, contributing liquidity to the same unified AMM that EVM users trade against. One price per outcome per event — Sui and EVM see the same depth.

Why this first: the EVM home-market stack is already built and deployed (Phase 2 of the EVM roadmap). Adding Sui as a proxy reuses that infrastructure — the AMM, the oracle resolver, the collateral accounting — without rebuilding it in Move. Sui becomes another spoke in the hub-and-spoke topology anchored on the EVM home.

1.2 Sui as a HOME chain (future path)

- **Home market:** lives on Sui, implemented natively in Move (see 01-move-market-contracts.md) with its object-centric AMM, Coin-based collateral, and oracle module.
- **EVM proxies:** lightweight Solidity `MarketProxy` contracts on Arc/Base/Ethereum/Polygon/BSC that forward intents to the Sui home market via Wormhole, receive confirmations, and mint wrapped position receipts representing Sui-native outcome shares.
- **Result:** Sui hosts the canonical market state and collateral pool; EVM users bridge in to trade against a Move-based AMM.

Why later: this path requires (a) the full Move market stack (AMM, factory, oracle) to be built and audited on Sui, (b) Wormhole adapters on both Sui and EVM sides tested for Sui→EVM message flow, and (c) operational confidence in Sui's finality and object model for cross-chain settlement. The Sui proxy path (1.1) validates the bridge architecture first with lower risk.

This document focuses on the Sui-as-proxy direction (1.1) as the initial design. The architecture is symmetric — the same patterns apply to Sui-as-home with minor role reversals.

---

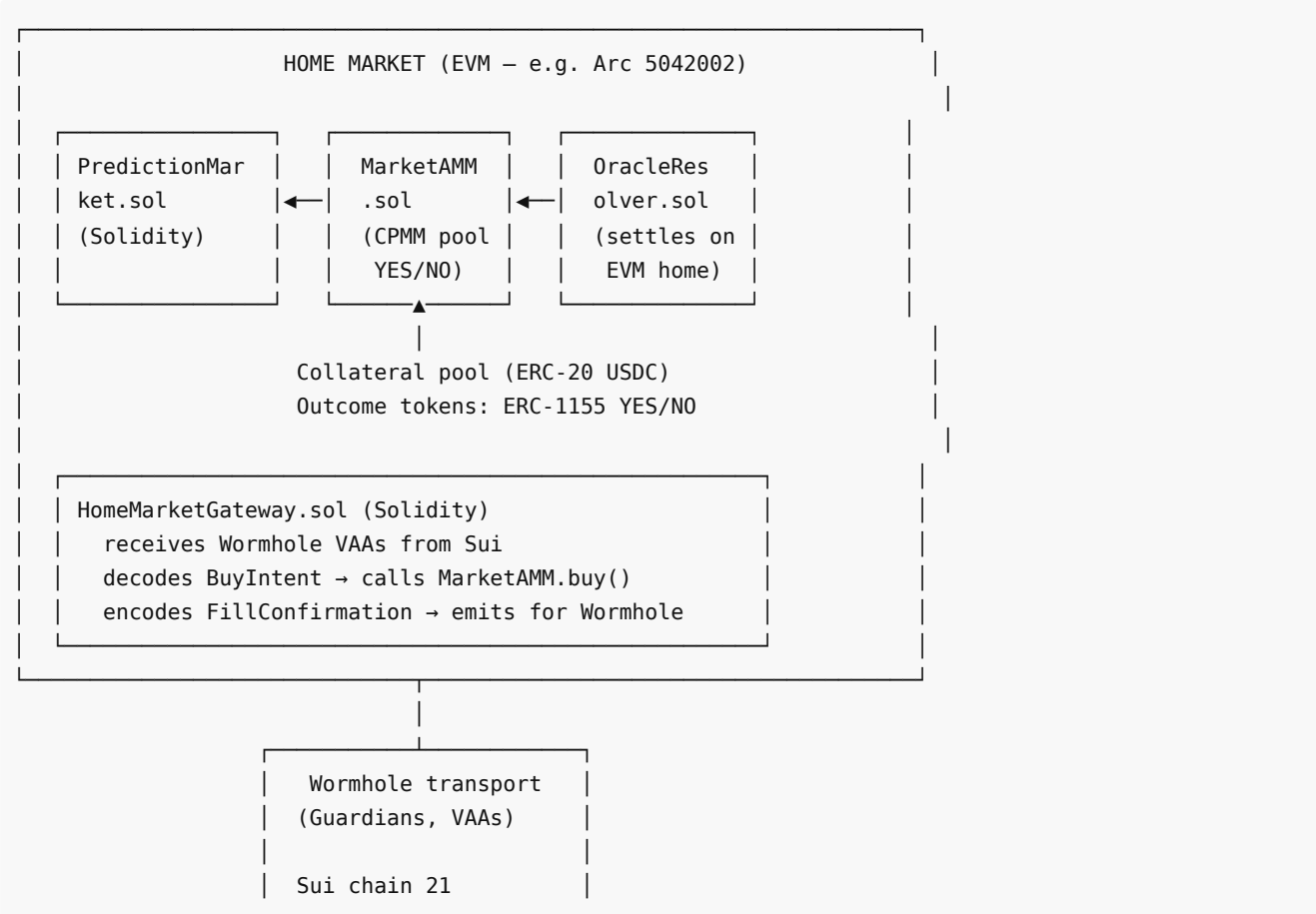
2. Architecture — mirroring the EVM bridge with a non-EVM adapter

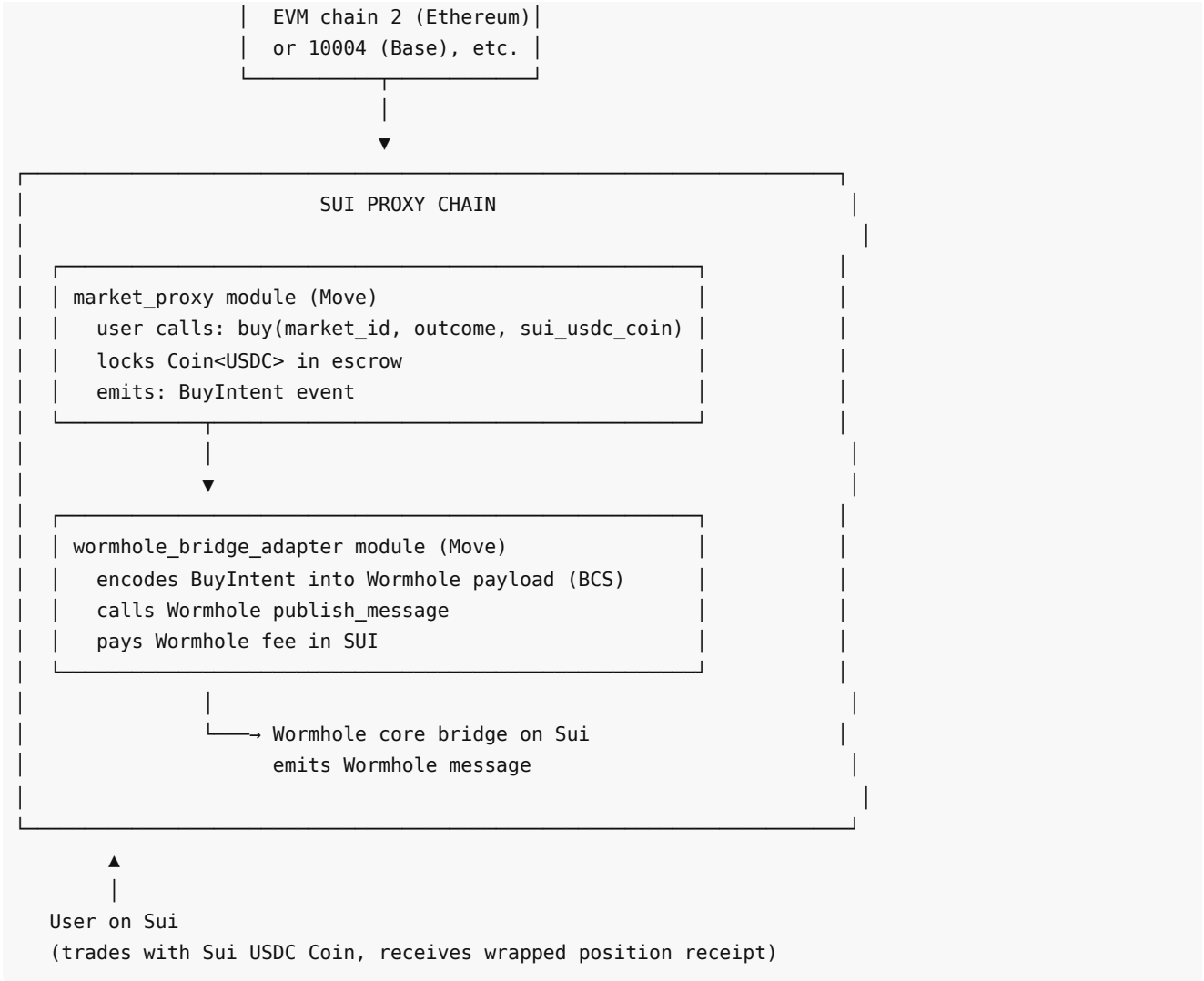
The Sui bridge keeps the **same architectural pattern as the EVM bridge** (which is inspired by Chainlink CCIP — see `documentation_bridge/00-overview.md` §3). The layers:

Layer	EVM bridge (CCIP)	Sui bridge (Wormhole)	Role

Chain-local entrypoint	`MarketProxy.sol` (Solidity)	`market_proxy` module (Move on Sui)	The user-facing contract/module on the proxy chain. Accepts local collateral (USDC/Coin), locks it, emits bridge intents.
Adapter interface / seam	`IBridgeAdapter` (Solidity interface)	`IBridgeAdapter`-equivalent trait in Move	Abstract send/receive interface decoupling the entrypoint from the concrete transport. One interface, many transports.
Concrete adapter (sender/receiver)	`CcipBridgeAdapter.sol` (wraps Chainlink CCIP Router)	`WormholeBridgeAdapter` module (wraps Wormhole `TokenBridge` and `Messenger`)	Transport-specific implementation: encodes messages on source, relays via the transport, decodes on destination.
Message transport	Chainlink CCIP DON (decentralized oracle network)	Wormhole Guardians (19-node Guardian network, 13-of-19 VAA consensus)	Off-chain infrastructure that observes source-chain events, reaches consensus, and delivers messages to the destination chain.
Risk / circuit-breaker layer	Chainlink RMN (Risk Management Network)	Wormhole's own Guardian quorum + Justify's per-route exposure caps	Independent monitoring layer that can halt message execution on anomaly detection. Justify's circuit breaker (see doc 04) wraps the Guardian security.

2.1 ASCII diagram — Sui proxy to EVM home





2.2 Resolution flow — EVM home to Sui proxy

```
HOME (EVM): OracleResolver.resolve(marketId, outcome) → emits MarketResolved

        │
        ▼
    Bridge dispatcher (watches MarketResolved on EVM)
        │
        └─→ HomeMarketGateway emits ResolutionBroadcast
                │
                ▼
           Wormhole Guardians
        (VAA signed 13-of-19)
                │
                ▼
        Sui: wormhole_bridge_adapter.receive_resolution(vaa)
                │
                └─→ market_proxy.mark_resolved(market_id, outcome)
                        │
                        └─→ users call market_proxy.redeem(receipt_id)
                               burns wrapped receipt
                               transfers Sui USDC Coin to user
```

---

3. Chainlink-CCIP-inspired layering — explicit and retained across the EVM/non-EVM boundary

Acknowledge the inspiration, explicitly: The Sui bridge architecture is **directly inspired by the Chainlink CCIP layering model** — the same separation of chain-local entrypoint, per-route adapter seam, message-carrying transport, and independent risk layer that the EVM bridge uses (see `documentation_bridge/00-overview.md` §3). This is **by design, not coincidence**: Justify adopts Chainlink's proven architectural pattern as the foundation for all cross-chain integrations, EVM and non-EVM alike.

3.1 The pattern (CCIP-inspired, transport-agnostic)

Architectural layer	CCIP term (EVM)	Sui bridge equivalent	Responsibility
Entrypoint	Router	`market_proxy` module	User-facing entry point on the proxy chain. Accepts collateral, locks it, emits intents.
Adapter seam	IRouterClient / CCIPReceiver	`IBridgeAdapter` trait (Move)	Abstract interface decoupling entrypoint from transport. `send_intent()`, `receive_confirmation()`, `receive_resolution()` methods.
Concrete adapter	OnRamp / OffRamp + CCIP Router	`wormhole_bridge_adapter` (Move + Solidity)	Transport-specific logic: encodes payloads (BCS on Sui, abi.encode on EVM), calls Wormhole, verifies VAAs, decodes on destination.
Transport / consensus	CCIP DON (Decentralized Oracle Network)	Wormhole Guardians (19-node network, 13-of-19 quorum)	Off-chain message relay: observes source-chain events, reaches consensus (VAA signature), delivers to destination.
Risk management	RMN (Risk Management Network)	Wormhole Guardian quorum + Justify circuit breaker	Independent verification layer that can halt execution on anomaly detection. Justify's per-route exposure caps wrap the underlying transport security.

3.2 The difference: concrete transport, not the pattern

The **architectural pattern** is identical between the EVM bridge and the Sui bridge. The **difference** is the **concrete transport** — the technology that actually carries messages:

Aspect	EVM bridge (CCIP)	Sui bridge (Wormhole)	Why the difference
Transport	Chainlink CCIP (Router, OnRamp, OffRamp, DON, RMN)	Wormhole (Guardians, VAAs, TokenBridge, Messenger modules)	CCIP has no mature Sui lane as of 2026-06; Sui is non-EVM, Move-native. Wormhole has first-class Sui support (Move SDK, Sui-native modules).
Consensus model	CCIP DON (decentralized oracle committee) commits Merkle roots; RMN blesses/curses	Wormhole Guardians (19 nodes, 13-of-19 threshold multisig) sign VAAs (Verifiable Action Approvals)	Both are decentralized quorum models; CCIP uses Merkle roots, Wormhole uses threshold signatures. Security profiles differ in details (see doc 04), but both are production-grade.
Message finality	CCIP waits for source-chain probabilistic finality	Wormhole Guardians observe Sui finality (~400ms) or EVM finality per chain;	Sui's single-leader consensus finalizes faster than EVM chains; Wormhole exploits this. Bridge time Sui→EVM can

	(e.g., 15 blocks) before DON commits	VAA issued after confirmation	be <1 min vs CCIP's 10–15 min for EVM→EVM.
Pluggability	`IBridgeAdapter` interface allows swapping CCIP for other transports (LayerZero, Wormhole)	`IBridgeAdapter` trait allows swapping Wormhole for a future Sui CCIP lane once available	The **seam is the same** — Move's trait system plays the role of Solidity's interface. The adapter is a parameter, not a commitment.

Key point: When Chainlink CCIP adds a mature Sui lane (Sui testnet/mainnet support, production DON, RMN coverage), Justify can swap `WormholeBridgeAdapter` for `CcipBridgeAdapter` **without changing the `market_proxy` entrypoint or the message payloads** — just deploy a new adapter that implements the same `IBridgeAdapter` trait. The architecture is **transport-agnostic by design**, even though the initial deployment uses Wormhole.

3.3 Why the CCIP-inspired pattern matters for Sui

1. **Separation of concerns:** The `market_proxy` module knows nothing about Wormhole — it only knows the adapter trait. Swapping transports (Wormhole → future Sui CCIP) is an adapter replacement, not a protocol rewrite.
2. **Defense in depth:** Justify's circuit breaker (per-route exposure caps, pause on anomaly) wraps the Wormhole Guardian security, the same way the EVM bridge's circuit breaker wraps CCIP's RMN. The adapter is trusted-but-verified.
3. **Unified architecture:** Engineers working on the EVM bridge and the Sui bridge see the **same conceptual model** — entrypoint, adapter seam, transport, risk layer. The only difference is Move vs Solidity syntax and Wormhole vs CCIP concretely. This lowers cognitive load and design surface.
4. **Proven at scale:** CCIP's architecture has secured billions in cross-chain value. By adopting its pattern (not reimplementing CCIP, but mirroring its layering), Justify inherits design rigor tested in production.

Honest statement: Justify's Sui bridge is **not a Chainlink CCIP integration** (because CCIP doesn't support Sui lanes yet), but it is **a Chainlink-CCIP-inspired architecture using Wormhole as the concrete transport**. The seam (`IBridgeAdapter`), the entrypoint pattern, and the risk-management layering are all lifted from the CCIP model. Wormhole is the best available transport for Sui today; the architecture is ready to adopt CCIP once Sui lanes are production-ready.

---

4. Why Wormhole for Sui (and how it differs from CCIP)

4.1 Wormhole's Sui support

Wormhole is a general-message bridge with first-class support for **Sui and Move**:

- **Sui-native modules:** Wormhole's Sui integration includes Move modules (`wormhole::core` , `wormhole::token_bridge` , `wormhole::messenger`) deployed on Sui mainnet and testnet. These are not EVM ports; they are idiomatic Move code using Sui's object model.
- **VAAs (Verifiable Action Approvals):** Wormhole's consensus primitive is a signed attestation (VAA) produced by the Guardian network. A VAA is a cryptographic proof that "this message was emitted on chain X and observed/signed by 13 of 19 Guardians." The destination chain verifies the VAA signatures on-chain before executing the message.
- **Guardian network:** 19 independent nodes run by institutions (Jump, Coinbase, Staked, etc.). 13-of-19 threshold for VAA issuance — Byzantine-fault-tolerant as long as fewer than 7 Guardians are compromised.
- **Sui chain ID:** Wormhole assigns Sui chain ID **21**. EVM chains have their own Wormhole IDs (Ethereum=2, BSC=4, Polygon=5, Base=10004, Arc=custom if supported). The adapter translates between Justify's EVM chain IDs and Wormhole IDs.

Key difference from CCIP:

Aspect	Chainlink CCIP	Wormhole	Implication for Justify
Consensus primitive	Merkle root commitment + DON attestation	Threshold-signed VAA (13-of-19 Guardians)	Wormhole VAAs are self-contained (no on-chain Merkle tree verification); CCIP is Merkle-proof-based.

Finality model	CCIP waits for source-chain finality	Wormhole waits for finality per chain config	Both safe; Wormhole's latency on Sui→EVM can be faster (Sui finalizes ~400ms).
Message format	`Client.EVM2AnyMessage` (ABI-encoded)	BCS (Binary Canonical Serialization) on Sui, ABI-encoded on EVM	Cross-boundary messages must transcode: BCS on Sui side, abi.encode on EVM side. Adapter handles this.
Token transfers	CCIP Token Pools (lock-mint or burn-mint)	Wormhole TokenBridge (lock-mint or native pools)	Both support USDC bridging. Wormhole integrates with Circle CCTP on supported lanes for native USDC minting.
Risk management	Independent RMN layer (bless/curse)	Guardian quorum (no separate RMN equivalent)	Justify's circuit breaker (exposure caps, pause) wraps Guardian security — similar to how it wraps CCIP RMN.

4.2 Why Wormhole is the right choice today

1. **Sui support exists:** Wormhole has production-deployed Move modules on Sui mainnet. CCIP does not (as of 2026-06).
2. **Move-native:** Wormhole's Sui SDK is idiomatic Move (objects, `Coin<T>`, `TxContext`), not a Solidity mindset forced onto Move. This makes the adapter cleaner.
3. **Battle-tested on Sui:** Wormhole has bridged billions through Sui (Portal Bridge for tokens, cross-chain apps like Mayan Finance). The Guardian network has Sui finality figured out.
4. **General-purpose messaging:** Wormhole supports arbitrary payloads (not just token transfers), which Justify needs for `BuyIntent`, `FillConfirmation`, `ResolutionBroadcast` messages.
5. **CCTP integration:** Wormhole integrates with Circle CCTP on EVM lanes, so USDC can move Sui→EVM as native USDC (not wrapped), the same as the EVM bridge's CCIP+CCTP strategy.

The trade-off: Wormhole's Guardian model is a 19-node multisig (albeit a large, geographically distributed one), whereas CCIP's DON + RMN is a two-layer decentralized consensus model. Justify's circuit breaker (per-route exposure caps, governance pause) mitigates this — if the Guardian set is compromised, the blast radius is capped by per-route limits (see 04-sui-bridge-security.md).

4.3 Future: Sui CCIP lane

When Chainlink ships a production Sui CCIP lane (Sui Router, OnRamp/OffRamp contracts in Move, DON support for Sui finality, RMN coverage), Justify can:

1. Implement a `CcipBridgeAdapter` (Move module) that wraps the Sui CCIP Router, matching the `IBridgeAdapter` trait.
2. Deploy the CCIP adapter alongside the Wormhole adapter (same seam, different transport).
3. Let governance/users choose per market: "Bridge via CCIP" or "Bridge via Wormhole."
4. Deprecate Wormhole for new markets once CCIP coverage reaches feature parity.

The adapter seam makes this swap architectural, not surgical. The `market_proxy` module doesn't change; only the injected adapter changes.

---

5. Core flows — Sui proxy to EVM home (conceptual)

Detailed flow diagrams are in 03-wormhole-adapter-and-interop.md. High-level:

5.1 Cross-chain buy (Sui user → EVM home market)

1. **User on Sui:** calls `market_proxy::buy(market_id, outcome_index, sui_usdc_coin, max_slippage)`.
2. **Proxy locks collateral:** transfers the `Coin<USDC>` into an escrow object on Sui, records a pending intent.
3. **Intent bridged to EVM home:** `wormhole_bridge_adapter` encodes the intent (`BuyIntent` struct) into BCS, calls `wormhole::publish_message()` on Sui, pays fee in SUI. Wormhole Guardians observe the message, sign a VAA.

4. **EVM home receives intent:** `HomeMarketGateway.sol` on the EVM chain (e.g., Arc) calls `wormhole::parseAndVerifyVM()` to validate the VAA, decodes the BCS payload into Solidity types, calls `MarketAMM.buy()` against the unified pool.
5. **Fill confirmation bridged back:** `HomeMarketGateway` emits `FillConfirmation`, which Wormhole Guardians observe and sign as a return VAA. The Sui `wormhole_bridge_adapter` receives the VAA, verifies it, decodes the confirmation.
6. **Proxy mints wrapped receipt:** `market_proxy` mints a Sui object (an outcome position receipt) to the user's address. The user's USDC is locked on Sui; the receipt entitles them to redeem for USDC (or nothing) after resolution, based on the EVM home market's outcome.

Result: Sui user contributes USDC to the EVM AMM pool without leaving Sui. The AMM depth is unified — Sui users and EVM users trade against the same reserves.

5.2 Cross-chain sell (future — detail TBD)

Symmetric to buy: user on Sui burns their wrapped receipt, `market_proxy` emits `SellIntent`, EVM home sells the underlying shares, collateral unlocked on Sui.

5.3 Cross-chain resolution (EVM home → Sui proxy)

1. **EVM home resolves:** `OracleResolver.resolve(marketId, outcome)` on the EVM chain (called by the CRE resolver or manual operator). Emits `MarketResolved`.
2. **Resolution broadcast:** Bridge dispatcher (off-chain service watching `MarketResolved`) calls `HomeMarketGateway.broadcastResolution(marketId, outcome)`. Wormhole Guardians sign a VAA.
3. **Sui proxy receives resolution:** `wormhole_bridge_adapter` on Sui verifies the VAA, decodes the `ResolutionBroadcast`, calls `market_proxy::mark_resolved(market_id, outcome)`.
4. **Users redeem on Sui:** `market_proxy::redeem(receipt_id)` burns the wrapped receipt, transfers the locked USDC to the user if their outcome won, else no payout.

Property: One resolution on the EVM home chain settles every proxy (Sui + all EVM proxies). No proxy can disagree with home state.

---

6. Differences vs the EVM bridge — forced by the non-EVM boundary

Aspect	EVM bridge (CCIP, EVM↔EVM)	Sui bridge (Wormhole, Sui↔EVM)	Why the difference
Bytecode/ABI	Both chains speak Solidity ABI; payloads are `abi.encode(...)`	Sui uses BCS (Binary Canonical Serialization); EVM uses ABI-encode	No shared type system across Move and Solidity. Messages must be serialized to a chain-neutral format (BCS on Sui side, decoded to Solidity types on EVM side, or vice versa).
Address format	20-byte `address` on all EVM chains	Sui uses 32-byte object IDs and account addresses	Adapter must map or embed addresses. E.g., Sui user address is hashed/encoded into the Wormhole payload; EVM side decodes it as `bytes32`, not `address`.
Collateral type	ERC-20 USDC on all EVM chains	Sui uses `Coin<USDC>` (Sui's native coin standard)	Wormhole TokenBridge or CCTP maps Sui USDC Coin ↔ EVM ERC-20 USDC. The value leg is separate from the message leg (same as the EVM bridge's CCIP+CCTP split).
Finality model	EVM finality (PoS epochs, OP Stack L1 finality, etc.)	Sui: single-leader BFT, ~400ms finality	Wormhole waits for Sui finality before Guardians sign VAA; latency Sui→EVM can be faster than EVM→EVM CCIP. Sui users may see fills in <1 min vs 10–15 min for EVM users bridging via CCIP.

Smart contract language	Solidity on both sides	Move on Sui, Solidity on EVM	Adapter logic must be written twice: `wormhole_bridge_adapter.move` on Sui, `HomeMarketGateway.sol` on EVM. The **seam** (`IBridgeAdapter`) is the same conceptually, syntax differs.
Message transport	CCIP Router, OnRamp, OffRamp (EVM-native contracts)	Wormhole core bridge (Move module on Sui, Solidity on EVM)	Wormhole's cross-VM design handles the Sui/EVM boundary. CCIP's current architecture is EVM-only (no Move support yet).

6.1 Serialization: BCS ↔ ABI-encode

BCS (Binary Canonical Serialization) is Move's canonical serialization format (like Protobuf or RLP). Sui contracts emit BCS-encoded structs; the Wormhole payload carries raw bytes.

On the EVM side: `HomeMarketGateway.sol` receives the Wormhole VAA, extracts the payload (bytes), and **decodes BCS by hand** (or uses a Solidity BCS library if available) into Solidity types (`uint256 marketId` , `uint8 outcome` , etc.).

On the Sui side: `wormhole_bridge_adapter.move` receives the VAA, extracts the payload, decodes it from ABI-encoded bytes (if coming from EVM) into Move structs using a BCS deserializer.

Implication: The adapter on each side is responsible for serialization/deserialization. The message schema (field order, types, encoding) is a **frozen integration contract** documented in 03-wormhole-adapter-and-interop.md.

6.2 Address translation

Sui addresses are 32 bytes; EVM addresses are 20 bytes. Wormhole payloads use **32-byte** `bytes32` as the universal address format:

- **Sui → EVM:** Sui user's address (32 bytes) is passed as-is in the Wormhole payload. EVM side stores it as `bytes32` , uses it as the key for wrapped receipts (or hashes it to 20 bytes if an EVM address is needed — design choice per 03-wormhole-adapter-and-interop.md).
- **EVM → Sui:** EVM user's address (20 bytes) is left-padded to 32 bytes in the Wormhole payload. Sui side truncates or stores as `vector<u8>` .

6.3 Value leg: Coin vs ERC-20

Sui collateral is `Coin<USDC>` , not ERC-20. The value leg is handled by:

1. **Wormhole TokenBridge** (lock-mint model): user locks `Coin<USDC>` on Sui, Wormhole mints wrapped USDC on EVM, or vice versa.
2. **Circle CCTP** (burn-mint native USDC): where CCTP supports Sui (future — CCTP is adding Sui testnet support as of mid-2026), native Sui USDC can be burned on Sui, minted as native EVM USDC on the destination.

Current recommendation: Wormhole TokenBridge for MVP (supported today), migrate to CCTP once Sui CCTP lanes are production-ready. The adapter seam abstracts this; the `market_proxy` doesn't care which bridge moves the USDC.

6.4 Object-centric receipts

On Sui, the wrapped position receipt is a **Sui object** (not a fungible token ID). The `market_proxy` module mints an object (e.g., `PositionReceipt { market_id, outcome, shares, owner }`) and transfers it to the user's address. Redemption burns the object.

On EVM, wrapped receipts are ERC-1155 or ERC-20 tokens. The semantic is the same (proof of claim on home-chain shares), but the implementation differs due to Sui's object model.

---

7. Glossary delta — new terms for the Sui bridge

These terms supplement the EVM bridge glossary (`documentation_bridge/00-overview.md` §5).

Term	Definition
VAA	Verifiable Action Approval — Wormhole's consensus primitive. A threshold-signed message (13-of-19 Guardians) attesting that an event occurred on a source chain. The destination chain verifies VAA signatures on-chain before executing.
Guardian	One of 19 independent nodes in the Wormhole network that observe cross-chain messages and sign VAAs. Threshold: 13-of-19 required for a VAA to be valid.
Wormhole chain ID	Wormhole-specific chain identifier (distinct from EVM chain IDs). Sui = 21, Ethereum = 2, BSC = 4, Polygon = 5, Base = 10004. Adapter maintains a mapping registry.
Emitter	The source contract/module that publishes a Wormhole message. On Sui: the `wormhole_bridge_adapter` module. On EVM: the `HomeMarketGateway` contract.
BCS	Binary Canonical Serialization — Move's standard serialization format (like Protobuf or RLP). Sui contracts emit BCS-encoded structs; the Wormhole payload carries BCS bytes decoded on the EVM side.
Coin	Sui's native fungible token standard. `Coin<USDC>` is Sui's representation of USDC. Wormhole TokenBridge or CCTP maps `Coin<USDC>` ↔ ERC-20 USDC on EVM chains.
Object (Sui)	Sui's first-class blockchain primitive. A wrapped position receipt on Sui is an object (not a token ID). Objects are owned by addresses, transferred via `transfer::transfer`, and can be destroyed (burned).
HomeMarketGateway	Solidity contract on the EVM home chain that receives Wormhole VAAs from Sui, decodes intents, calls the AMM, and emits fill confirmations back to Sui. The Sui↔EVM translation layer.

---

8. Forward links

The Sui bridge design is detailed across four documents (including this overview):

#	Document	What it covers
02	**02-sui-bridge-overview.md** (this document)	The problem (Sui liquidity isolation), the home/proxy model on Sui, Chainlink-CCIP-inspired layering, Wormhole as the concrete transport.
03	**03-wormhole-adapter-and-interop.md**	The Wormhole adapter (Move + Solidity sides), VAA structure, Sui↔EVM message and value flow, BCS↔ABI-encode transcoding, wrapped receipts.
04	**04-sui-bridge-security.md**	Security model: Wormhole Guardians vs Chainlink RMN, per-route exposure caps, circuit breaker, object-ownership safety, finality guarantees.

Additionally, the Move market contracts (the Sui-as-home path, future) are covered in:

#	Document	What it covers
00	**00-sui-polymarkets-overview.md**	Why Sui; the object-centric prediction-market model; product parity with the EVM stack.
01	**01-move-market-contracts.md**	The Move package: factory, AMM, outcome coins, oracle — object-model design.

---

9. Status and integration with the Justify roadmap

Current phase (as of 2026-06-13): Phase 1 (Arc testnet deployment) is in progress for the **EVM stack**. The Sui bridge is a **separate project** — designed forward-looking, not yet deployed.

Sui bridge phase: This documentation set defines the target architecture for integrating Sui markets into the Justify network. The work sequences as:

1. **Phase 1 (EVM-only, separate):** Deploy the Move market stack on Sui independently (local Sui testnet, seeded markets, manual oracle). Validate the Move AMM math, object-model safety, `Coin<USDC>` accounting. No bridge yet.
2. **Phase 2 (Sui proxy to EVM home):** Build the Wormhole adapter (Sui + EVM sides), deploy `market_proxy` on Sui, stand up one test lane (Sui testnet → Base testnet or Arc testnet). End-to-end: Sui user buys a share on an EVM home market, receives wrapped receipt, redeems after resolution. This is the **Sui-as-proxy path** (§1.1).
3. **Phase 3 (Sui as home, EVM proxies):** Flip the direction: deploy a Move market on Sui mainnet as the home, deploy Solidity `MarketProxy` contracts on Base/Ethereum/Polygon, bridge EVM intents to the Sui AMM. This is the **Sui-as-home path** (§1.2), unlocking Sui as a first-class home chain in the Justify network.

Design freeze: These documents define the target so that when Phase 2 begins, the `IBridgeAdapter` trait, the Wormhole message schema, and the `market_proxy` ↔ EVM home integration can be built against a stable specification. The interfaces and payloads are the **integration contract** between the Sui bridge and the EVM home stack.

Related documentation:

- **Whitepaper §8** (`documentation_justify_whitepaper/justify-whitepaper.md`) — the "chain is a parameter" claim extended to non-EVM chains.
- **EVM bridge set** (`documentation_bridge/`) — the home/proxy model, `IBridgeAdapter` , Chainlink CCIP adapter, resolution propagation — the patterns this Sui bridge mirrors.
- **CRE resolver plan** (`docs/delivery/cre-resolver-plan.md`) — the Chainlink-based automated resolver that, in the bridge phase, settles the EVM home market once and propagates to Sui (and all EVM proxies).

---

10. Properties and trade-offs recap

Property	Consequence / design choice
Unified liquidity	Sui users and EVM users trade against the **same AMM pool** (on the home chain). One price per outcome per event — no Sui-vs-EVM arbitrage, no liquidity fragmentation by VM.
Single source of truth	The home market (EVM or Sui, depending on direction) resolves once; proxies mirror that resolution. Sui proxies cannot disagree with an EVM home's outcome.
Asynchronous fills	A bridged buy from Sui confirms in Wormhole time (~1–5 min Sui→EVM, depending on EVM finality). UI quotes a guaranteed-bounds fill (max slippage) at intent time; home AMM fills within it or refunds.
Bridge trust	A Sui proxy position is as safe as min(Sui security, EVM home security, Wormhole Guardian quorum). Per-route exposure caps limit blast radius if Guardians are compromised (see doc 04).
Transport pluggability	The `IBridgeAdapter` trait (Move equivalent of the EVM bridge's Solidity interface) lets Justify swap Wormhole for a future Sui CCIP lane without changing `market_proxy` or the message schemas.
Cross-VM message format	BCS on Sui, ABI-encode on EVM. Adapters handle transcoding. The payload schema (field order, types) is frozen and versioned — a breaking change requires a new adapter deployment or migration.
Sui finality advantage	Sui's ~400ms finality means Sui→EVM bridge time can be faster than EVM→EVM CCIP (which waits for EVM finality ~15 min). Sui users may see sub-minute fills vs 10–15 min for EVM users.
Object-centric receipts	Wrapped position receipts on Sui are Sui objects (not ERC-1155 token IDs). This leverages Sui's ownership model for safety — the receipt can't be "approved" away like an ERC-20; transfer is explicit and safe.

Progressive rollout	Sui-as-proxy first (reuses EVM home stack), Sui-as-home later (requires Move AMM audit, operational confidence). The architecture supports both in parallel.

---

11. Next steps

Readers new to the Sui bridge should proceed in document order:

1. **03-wormhole-adapter-and-interop.md** — the Wormhole adapter (Move + Solidity), VAA structure, message and value flow, BCS ↔ ABI-encode transcoding.
2. **04-sui-bridge-security.md** — Wormhole Guardians vs Chainlink RMN, per-route exposure caps, circuit breaker, object-ownership safety.

Engineers building the Sui bridge should treat the `IBridgeAdapter` trait (Move equivalent of the Solidity `IBridgeAdapter` interface) and the Wormhole message schema (doc 03) as the frozen integration contract.

Relation to the EVM bridge: The Sui bridge is **architecturally parallel** to the EVM bridge. The **pattern** (entrypoint → adapter seam → transport → risk layer) is identical, inspired by Chainlink CCIP. The **difference** is the concrete transport (Wormhole for Sui vs CCIP for EVM) and the cross-VM translation (BCS ↔ ABI-encode, Coin ↔ ERC-20). When Chainlink ships a Sui CCIP lane, Justify can adopt it by swapping the Wormhole adapter for a CCIP adapter — no change to the `market_proxy` module or the message payloads.

End of overview. Proceed to `03-wormhole-adapter-and-interop.md` for the Wormhole adapter specification and Sui ↔ EVM message flow.

Wormhole Adapter and Interop — The Sui ↔ EVM Bridge Transport

Document: 03-wormhole-adapter-and-interop.md
Status: Design (Sui project — separate from EVM line)
Related: 02-sui-bridge-overview.md | documentation_bridge/01-bridge-adapter-interface.md | documentation_bridge/02-chainlink-ccip-adapter.md

---

1. Wormhole primer — cross-chain messaging for non-EVM chains

Wormhole is a general-purpose cross-chain messaging protocol designed to connect heterogeneous blockchain ecosystems — EVM (Ethereum, Base, Polygon, BSC), non-EVM (Sui, Aptos, Solana), and beyond. Where Chainlink CCIP is optimized for EVM ↔ EVM messaging with DON-based attestation, Wormhole is built for **multi-ecosystem interoperability** with a **Guardian network** that observes and signs cross-chain messages as **VAAs** (Verified Action Approvals).

Wormhole is the production-proven bridge for Sui ↔ EVM interop today (2026-06).

1.1 Core Wormhole components

1.1.1 Core Bridge contract (per chain)

Every supported chain deploys a **Wormhole Core Bridge** contract (Solidity on EVM, Move on Sui, Rust on Solana) that serves as the single entrypoint for cross-chain messages.

On the source chain:

- Senders call `publishMessage(nonce, payload, consistencyLevel)` to emit a message.
- The core contract increments a **sequence number** per emitter address.
- Emits a `LogMessagePublished` event with `(emitter, sequence, payload, consistencyLevel)`.

On the destination chain:

- Receivers call `parseAndVerifyVAA(encodedVAA)` to validate an incoming message.
- The core contract verifies the VAA's Guardian signatures and marks it consumed (replay protection).
- Destination contracts extract the payload and execute business logic.

1.1.2 Guardian network

Wormhole's **Guardians** are a permissioned set of 19 validators (as of 2026) run by established entities (Jump Crypto, Chorus One, Figment, etc.). Guardians:

1. **Observe** `LogMessagePublished` events on all supported chains.
2. **Attest** to observed messages by signing a **VAA** (Verified Action Approval) — a data structure containing the message payload, source chain, emitter address, sequence number, and a threshold signature (13-of-19 Guardian consensus).
3. **Publish** VAAs to a gossip network where relayers and users can fetch them.

The Guardian network is **off-chain**; VAAs are cryptographic proofs that a message was observed and attested by a quorum of Guardians.

1.1.3 VAA (Verified Action Approval)

A **VAA** is the core Wormhole primitive — a signed attestation that a message was published on a source chain:

```
┌──────────────────────────────────────────────────────────────┐
│ VAA (Verified Action Approval)                               │
├──────────────────────────────────────────────────────────────┤
│ version: u8                                                  │
│ guardianSetIndex: u32       (which Guardian set signed this) │
```

```
| signatures: Vec<Signature>  (Guardian signatures; 13-of-19)      |
| timestamp: u32              (Guardian observation time)          |
| nonce: u32                  (publisher-chosen nonce)             |
| emitterChain: u16           (Wormhole chain ID, source)          |
| emitterAddress: [u8; 32]    (32-byte canonical address)          |
| sequence: u64               (monotonic per emitter)              |
| consistencyLevel: u8        (finality threshold)                 |
| payload: Vec<u8>            (arbitrary application data)         |
└──────────────────────────────────────────────────────────────────┘
```

Key properties:

- **Self-contained:** the VAA includes the entire message; no on-chain query needed.
- **Replay protection:** each VAA's hash is marked consumed on the destination chain after verification.
- **Ordering:** sequence numbers are per-emitter, enforcing message order from a single sender.

1.1.4 Wormhole chain IDs (distinct from native chain IDs)

Wormhole assigns a **uint16 chain ID** to each supported blockchain, distinct from native chain IDs:

Chain	Native Chain ID	Wormhole Chain ID
Ethereum	1	2
Base	8453	30
Polygon PoS	137	5
BSC	56	4
Sui	(no EVM ID)	21
Solana	(no EVM ID)	1

The adapter must translate between Justify's EVM chain IDs and Wormhole chain IDs.

1.1.5 Wormhole Token Bridge

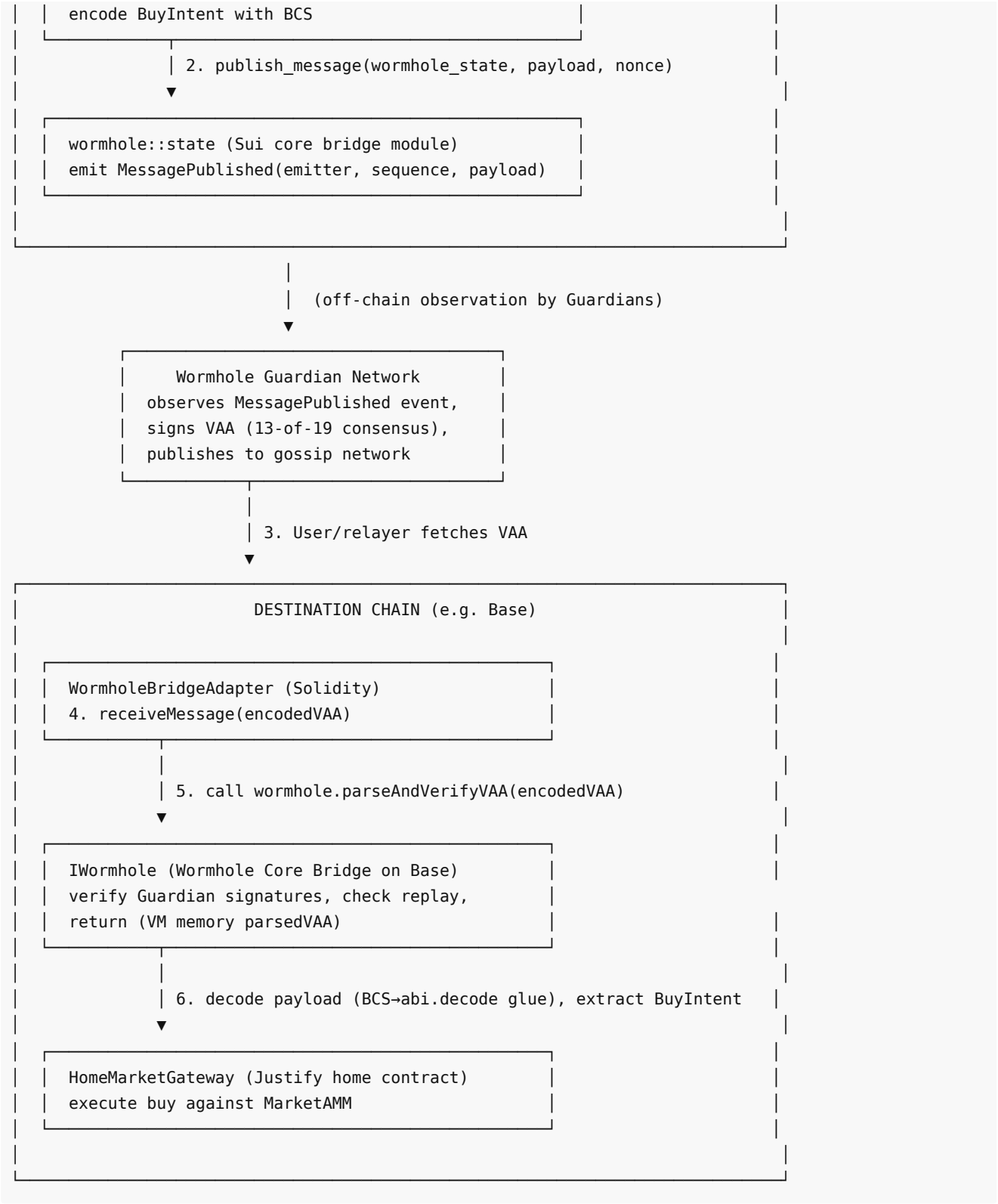
Wormhole provides a **Token Bridge** layer built on top of the core bridge for fungible token transfers:

- **Lock-and-mint** model: on the origin chain, tokens are locked in a custody contract; on the destination, wrapped representations are minted.
- **Attested tokens:** the Token Bridge tracks which tokens are "native" to which chain; wrapped tokens can be burned to unlock the native asset.

For USDC specifically, Circle's **CCTP** (Cross-Chain Transfer Protocol) is integrated with Wormhole on Sui and EVM chains that support native USDC. Where CCTP is available, use **Circle CCTP via Wormhole's CCTP relay** (burn native USDC, mint native USDC on destination) rather than the Token Bridge's wrapped model.

1.2 Message flow: Sui→EVM via Wormhole

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                          SOURCE CHAIN (Sui)                                  │
│                                                                              │
│  User calls Move entry function (e.g., market_proxy::buy)                   │
│       │                                                                      │
│       │ 1. Lock Coin<USDC> in MarketProxy shared object                      │
│       ▼                                                                      │
│  ┌──────────────────────────────────────────────────┐                        │
│  │  wormhole_adapter module (Move)                  │                        │
```



Key differences from CCIP:

- **No on-chain commit phase** — Wormhole VAAs are off-chain until submitted by a relayer or user; CCIP commits Merkle roots on-chain before execution.
- **User/relayer-driven delivery** — the VAA must be fetched from the Guardian network and submitted to the destination chain (permissionless); CCIP's Executing DON automatically delivers messages.
- **BCS↔ABI encoding boundary** — Sui uses BCS (Binary Canonical Serialization) for Move structs; EVM uses ABI encoding. The adapter must translate.

---

2. Sui side: Move wormhole_adapter module

2.1 Architecture

The `wormhole_adapter` module (part of the Justify Sui Move package) wraps the Sui Wormhole core bridge and provides:

1. **BUY flow:** accept user's `Coin<USDC>`, lock it in the MarketProxy, serialize a BuyIntent with BCS, publish a Wormhole message.
2. **Incoming VAA verification:** when a FillConfirmation or ResolutionBroadcast VAA arrives from an EVM chain, parse and verify the VAA, decode the payload, and execute the action (mint wrapped receipt, mark resolved).

2.2 Illustrative Move code

```
// File: sources/wormhole_adapter.move
module justify::wormhole_adapter {
    use sui::object::{Self, UID};
    use sui::tx_context::{Self, TxContext};
    use sui::coin::{Self, Coin};
    use sui::transfer;
    use sui::event;
    use sui::bcs;
    use wormhole::state::{State as WormholeState};
    use wormhole::publish_message;
    use wormhole::vaa::{Self, VAA};
    use wormhole::bytes::{Self as wormhole_bytes};

    /// Emitter capability — only the adapter can emit Wormhole messages for Justify
    struct EmitterCap has key, store {
        id: UID,
    }

    /// Adapter state — tracks consumed VAA hashes (replay protection), destination routes
    struct AdapterState has key {
        id: UID,
        consumed_vaas: Table<vector<u8>, bool>,  // VAA hash → consumed flag
        allowed_emitters: Table<u16, vector<u8>>,  // Wormhole chain ID → allowed emitter address
        home_chain_id: u16,  // Wormhole chain ID for the EVM home chain (e.g. 30 for Base)
    }

    /// BuyIntent — matches the struct from documentation_bridge/01
    struct BuyIntent has copy, drop {
        market_id: u64,
        trader: vector<u8>,  // 32-byte canonical address (Sui address)
        outcome_index: u8,
        collateral_in: u64,
        max_slippage: u64,
        nonce: u64,
    }

    /// FillConfirmation — incoming from EVM home market
    struct FillConfirmation has copy, drop {
        intent_message_id: vector<u8>,  // Original VAA hash
        market_id: u64,
        trader: vector<u8>,
        outcome_index: u8,
        shares_out: u64,
```

```
    execution_price: u64,
}

/// Events
struct MessagePublished has copy, drop {
    sequence: u64,
    nonce: u32,
    payload_len: u64,
}

struct VAAConsumed has copy, drop {
    vaa_hash: vector<u8>,
    emitter_chain: u16,
    sequence: u64,
}

/// Initialize the adapter (called once at deployment)
public fun init_adapter(
    wormhole_state: &WormholeState,
    home_wormhole_chain_id: u16,
    ctx: &mut TxContext
) {
    let emitter_cap = EmitterCap { id: object::new(ctx) };
    let adapter_state = AdapterState {
        id: object::new(ctx),
        consumed_vaas: table::new(ctx),
        allowed_emitters: table::new(ctx),
        home_chain_id: home_wormhole_chain_id,
    };
    transfer::share_object(adapter_state);
    transfer::transfer(emitter_cap, tx_context::sender(ctx));
}

/// Send a BuyIntent to the EVM home market
/// @param market_id: Home market ID on the EVM chain
/// @param outcome_index: 0 = YES, 1 = NO
/// @param collateral_coin: Locked USDC from the user
/// @param max_slippage: Basis points (e.g. 300 = 3%)
/// @param nonce: Per-user nonce for ordering
public entry fun send_buy_intent(
    adapter_state: &AdapterState,
    wormhole_state: &mut WormholeState,
    emitter_cap: &EmitterCap,
    market_id: u64,
    outcome_index: u8,
    collateral_coin: Coin<USDC>,
    max_slippage: u64,
    nonce: u64,
    ctx: &mut TxContext
) {
    let trader = tx_context::sender(ctx);
    let collateral_in = coin::value(&collateral_coin);

    // Lock collateral in the MarketProxy (omitted here; assume proxy holds coin)
    // transfer::public_transfer(collateral_coin, market_proxy_address);
```

```
        // Construct BuyIntent
        let intent = BuyIntent {
            market_id,
            trader: address::to_bytes(trader),
            outcome_index,
            collateral_in,
            max_slippage,
            nonce,
        };

        // Serialize with BCS
        let payload = bcs::to_bytes(&intent);

        // Publish Wormhole message
        // nonce: random u32 for uniqueness (can derive from Sui tx digest)
        let msg_nonce = (nonce % 0xFFFFFFFF) as u32;
        let consistency_level = 15u8;  // Sui finality (confirm with Wormhole Sui docs)

        let (sequence, _) = wormhole::publish_message::publish_message(
            wormhole_state,
            msg_nonce,
            payload,
            consistency_level,
            ctx
        );

        event::emit(MessagePublished {
            sequence,
            nonce: msg_nonce,
            payload_len: vector::length(&payload),
        });
    }

    /// Receive and verify a FillConfirmation VAA from the EVM home market
    /// @param encoded_vaa: VAA bytes fetched from Guardian network
    public entry fun receive_fill_confirmation(
        adapter_state: &mut AdapterState,
        wormhole_state: &WormholeState,
        encoded_vaa: vector<u8>,
        ctx: &mut TxContext
    ) {
        // Parse and verify VAA
        let vaa = wormhole::vaa::parse_and_verify(wormhole_state, encoded_vaa, ctx);

        // Replay protection: check VAA hash not consumed
        let vaa_hash = wormhole::vaa::hash(&vaa);
        assert!(!table::contains(&adapter_state.consumed_vaas, vaa_hash),
ERR_VAA_ALREADY_CONSUMED);
        table::add(&mut adapter_state.consumed_vaas, vaa_hash, true);

        // Validate emitter is allowlisted
        let emitter_chain = wormhole::vaa::emitter_chain(&vaa);
        let emitter_address = wormhole::vaa::emitter_address(&vaa);
        assert!(
            table::contains(&adapter_state.allowed_emitters, emitter_chain),
            ERR_EMITTER_NOT_ALLOWED
```

```
        );
        let allowed_emitter = table::borrow(&adapter_state.allowed_emitters, emitter_chain);
        assert!(emitter_address == *allowed_emitter, ERR_EMITTER_MISMATCH);

        // Extract payload and decode FillConfirmation
        let payload = wormhole::vaa::take_payload(vaa);
        let fill_confirmation: FillConfirmation = bcs::from_bytes(&payload);

        // Execute: mint wrapped receipt to trader on Sui
        // (MarketProxy logic omitted; assume proxy::mint_wrapped_receipt is called)
        // let trader_address = address::from_bytes(fill_confirmation.trader);
        // market_proxy::mint_wrapped_receipt(trader_address, fill_confirmation.outcome_index,
fill_confirmation.shares_out, ctx);

        event::emit(VAAConsumed {
            vaa_hash,
            emitter_chain,
            sequence: wormhole::vaa::sequence(&vaa),
        });
    }

    /// Admin function: allowlist an EVM emitter address for a Wormhole chain ID
    public entry fun allowlist_emitter(
        adapter_state: &mut AdapterState,
        wormhole_chain_id: u16,
        emitter_address: vector<u8>,
        ctx: &mut TxContext
    ) {
        // Access control omitted (only owner can call)
        table::add(&mut adapter_state.allowed_emitters, wormhole_chain_id, emitter_address);
    }

    // Error codes
    const ERR_VAA_ALREADY_CONSUMED: u64 = 1;
    const ERR_EMITTER_NOT_ALLOWED: u64 = 2;
    const ERR_EMITTER_MISMATCH: u64 = 3;
}
```

2.3 Key Move-specific patterns

- **Shared object `AdapterState`:** allows concurrent access for VAA verification across multiple transactions.
- **EmitterCap capability:** restricts who can publish messages on behalf of Justify (only the adapter module or authorized callers).
- **BCS serialization:** `bcs::to_bytes(&intent)` produces a canonical binary encoding that the EVM side must decode (see §4).
- **Wormhole `publish_message`:** returns `(sequence, ...)` for tracking; the sequence is per-emitter and monotonic.
- **VAA verification:** `wormhole::vaa::parse_and_verify()` checks Guardian signatures and finality; the module then enforces emitter allowlisting and replay protection.

---

3. EVM side: WormholeBridgeAdapter (Solidity)

3.1 Purpose

The `WormholeBridgeAdapter` on the EVM side (Base, Ethereum, etc.) implements the **same `IBridgeAdapter` interface** as the Chainlink CCIP adapter (from documentation_bridge/01-bridge-adapter-interface.md). This is the critical integration seam:

the `HomeMarketGateway` does not know whether messages arrive via CCIP or Wormhole — it calls the same `receiveMessage()` callback.

3.2 Illustrative Solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "./IBridgeAdapter.sol";
import "@wormhole-solidity-sdk/interfaces/IWormhole.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

/// @title WormholeBridgeAdapter
/// @notice Wormhole transport adapter for Justify Sui↔EVM bridge.
///         Implements the same IBridgeAdapter interface as ChainlinkCCIPAdapter,
///         enabling the HomeMarketGateway to remain transport-agnostic.
///
///         On SEND: encodes Justify message payloads (BuyIntent, etc.) into
///         abi.encodePacked format, publishes via Wormhole core bridge.
///
///         On RECEIVE: verifies incoming VAA from Wormhole core, decodes
///         BCS-encoded Sui payloads (via a BCS→ABI parser), and invokes
///         the destination contract's receiveMessage callback.
contract WormholeBridgeAdapter is IBridgeAdapter, Ownable {
    /// @notice Wormhole core bridge contract on this EVM chain
    IWormhole public immutable wormhole;

    /// @notice Mapping: Justify chain ID → Wormhole chain ID
    mapping(uint256 => uint16) public justifyToWormholeChain;

    /// @notice Mapping: Wormhole chain ID → Justify chain ID (reverse lookup)
    mapping(uint16 => uint256) public wormholeToJustifyChain;

    /// @notice Allowlisted remote emitter addresses per Wormhole chain ID
    mapping(uint16 => bytes32) public allowedEmitters;

    /// @notice Consumed VAA hashes (replay protection)
    mapping(bytes32 => bool) public consumedVAAs;

    /// @notice Destination contract for incoming messages (e.g., HomeMarketGateway)
    address public destinationReceiver;

    /// @notice Wormhole consistency level (finality threshold)
    uint8 public consistencyLevel = 15;

    event MessageSent(
        uint64 indexed sequence,
        uint16 indexed targetChain,
        bytes payload
    );

    event MessageReceived(
        bytes32 indexed vaaHash,
        uint16 indexed sourceChain,
        bytes32 indexed emitter,
        bytes payload
```

```
    );

    event EmitterAllowlisted(uint16 wormholeChainId, bytes32 emitterAddress);

    error VAAReplayed(bytes32 vaaHash);
    error EmitterNotAllowed(uint16 chainId, bytes32 emitter);
    error InvalidDestination();

    constructor(
        address _wormhole,
        address _destinationReceiver
    ) Ownable(msg.sender) {
        wormhole = IWormhole(_wormhole);
        destinationReceiver = _destinationReceiver;
    }

    /// ═══════════════════════════════════════════════════════════════════
    ///  IBridgeAdapter implementation — SEND
    /// ═══════════════════════════════════════════════════════════════════

    /// @notice Send a cross-chain message via Wormhole
    /// @param destChainSelector  Justify chain ID (translated to Wormhole chain ID)
    /// @param receiver           Destination contract address (encoded as bytes32 in Wormhole)
    /// @param payload            ABI-encoded message (BuyIntent | SellIntent | etc.)
    /// @param gasLimit           Ignored (Wormhole does not enforce destination gas limit)
    /// @return messageId         VAA hash (not available until Guardian signing; return sequence
as placeholder)
    function sendMessage(
        uint64 destChainSelector,
        address receiver,
        bytes calldata payload,
        uint256 gasLimit  // ignored for Wormhole
    ) external payable override returns (bytes32 messageId) {
        uint16 targetWormholeChain = justifyToWormholeChain[uint256(destChainSelector)];
        require(targetWormholeChain != 0, "WormholeAdapter: destination not configured");

        // Publish message via Wormhole core
        uint32 nonce = uint32(block.timestamp % type(uint32).max);  // pseudo-random nonce
        uint64 sequence = wormhole.publishMessage{value: msg.value}(
            nonce,
            payload,
            consistencyLevel
        );

        emit MessageSent(sequence, targetWormholeChain, payload);

        // Return sequence as messageId (VAA hash not yet available)
        messageId = keccak256(abi.encodePacked(sequence, targetWormholeChain));
    }

    /// @notice Quote the fee for a Wormhole message
    /// @dev Wormhole fee is the core bridge's messageFee(), independent of payload size
    function quoteFee(
        uint64 destChainSelector,
        address receiver,
        bytes calldata payload,
```

```
        uint256 gasLimit
    ) external view override returns (uint256 feeAmount) {
        return wormhole.messageFee();
    }

    /// ═══════════════════════════════════════════════════════════════════════
    ///  IBridgeAdapter implementation — RECEIVE
    /// ═══════════════════════════════════════════════════════════════════════

    /// @notice Receive and verify a Wormhole VAA, decode payload, forward to destination
    /// @param sourceChainSelector  Wormhole source chain ID (translated to Justify chain ID)
    /// @param sender               Emitter address on source chain (32-byte Wormhole canonical)
    /// @param payload              Encoded VAA bytes (full VAA including signatures)
    function receiveMessage(
        uint64 sourceChainSelector,  // Wormhole chain ID
        address sender,               // Ignored (we parse emitter from VAA)
        bytes calldata payload        // Encoded VAA
    ) external override {
        // Parse and verify VAA
        (IWormhole.VM memory vm, bool valid, string memory reason) =
wormhole.parseAndVerifyVM(payload);
        require(valid, string(abi.encodePacked("Invalid VAA: ", reason)));

        // Replay protection
        bytes32 vaaHash = vm.hash;
        if (consumedVAAs[vaaHash]) revert VAAReplayed(vaaHash);
        consumedVAAs[vaaHash] = true;

        // Validate emitter is allowlisted
        uint16 emitterChain = vm.emitterChainId;
        bytes32 emitter = vm.emitterAddress;
        bytes32 allowedEmitter = allowedEmitters[emitterChain];
        if (allowedEmitter == bytes32(0) || allowedEmitter != emitter) {
            revert EmitterNotAllowed(emitterChain, emitter);
        }

        // Decode the payload (BCS-encoded from Sui → decode to Justify structs)
        // For simplicity, assume payload is already ABI-encoded on Sui side (hybrid encoding)
        // Production: implement a BCS→ABI decoder library or off-chain relay that re-encodes
        bytes memory message = vm.payload;

        emit MessageReceived(vaaHash, emitterChain, emitter, message);

        // Forward to destination receiver (HomeMarketGateway or MarketProxy)
        if (destinationReceiver == address(0)) revert InvalidDestination();
        uint256 sourceJustifyChain = wormholeToJustifyChain[emitterChain];

        IBridgeReceiver(destinationReceiver).receiveMessage(
            uint64(sourceJustifyChain),
            address(uint160(uint256(emitter))),  // Convert bytes32 to address (lossy for Sui,
safe for EVM)
            message
        );
    }

    /// ═══════════════════════════════════════════════════════════════════════
```

```
    ///  Admin functions
    /// ═══════════════════════════════════════════════════════════════════

    /// @notice Configure Justify↔Wormhole chain ID mapping
    function configureChainMapping(
        uint256 justifyChainId,
        uint16 wormholeChainId
    ) external onlyOwner {
        justifyToWormholeChain[justifyChainId] = wormholeChainId;
        wormholeToJustifyChain[wormholeChainId] = justifyChainId;
    }

    /// @notice Allowlist a remote emitter (e.g., the Sui wormhole_adapter module)
    /// @param wormholeChainId  Wormhole chain ID (e.g., 21 for Sui)
    /// @param emitterAddress   32-byte canonical address of the emitter
    function allowlistEmitter(uint16 wormholeChainId, bytes32 emitterAddress)
        external
        onlyOwner
    {
        allowedEmitters[wormholeChainId] = emitterAddress;
        emit EmitterAllowlisted(wormholeChainId, emitterAddress);
    }

    /// @notice Update destination receiver (HomeMarketGateway, etc.)
    function setDestinationReceiver(address _receiver) external onlyOwner {
        destinationReceiver = _receiver;
    }

    /// @notice Set Wormhole consistency level (finality threshold)
    function setConsistencyLevel(uint8 _level) external onlyOwner {
        consistencyLevel = _level;
    }
}

/// @notice Callback interface for destination contracts receiving Wormhole messages
interface IBridgeReceiver {
    function receiveMessage(
        uint64 sourceChainSelector,
        address sender,
        bytes calldata payload
    ) external;
}
```

3.3 Key Solidity-specific patterns

- **IWormhole.parseAndVerifyVM:** the Wormhole core contract on EVM returns a `VM` (Verified Message) struct containing parsed VAA fields and verification status.
- **Replay protection via `consumedVAAs` mapping:** each VAA hash is marked consumed after processing.
- **Emitter allowlisting:** only VAAs from pre-configured Sui emitter addresses (the `wormhole_adapter` module) are accepted.
- **IBridgeAdapter interface compliance:** the same `sendMessage()` / `receiveMessage()` / `quoteFee()` signatures as the CCIP adapter, so `HomeMarketGateway` can swap adapters without code changes.

---

4. Message encoding across the Sui ↔ EVM boundary

4.1 The BCS↔ABI translation challenge

Sui Move serializes structs with **BCS** (Binary Canonical Serialization) — a compact, deterministic encoding designed for Move's type system. **EVM Solidity** uses **ABI encoding** (abi.encode / abi.decode) with different padding and type representations.

Problem: A `BuyIntent` struct serialized with BCS on Sui cannot be directly decoded with `abi.decode` on EVM.

Solutions:

Option 1: Hybrid encoding (staged rollout)

For MVP / Phase 3, the Sui side encodes messages in a **hybrid format** that mimics ABI encoding:

- Serialize each field as a fixed-width big-endian integer (u64 → 8 bytes, u8 → 1 byte, address → 32 bytes).
- Concatenate fields in the same order as the Solidity struct.
- EVM side uses `abi.decode` with explicit offsets.

Example (BuyIntent):

```
// Sui side (hybrid encoding for EVM compatibility)
public fun encode_buy_intent_abi(intent: &BuyIntent): vector<u8> {
    let mut buf = vector::empty<u8>();
    vector::append(&mut buf, bcs::to_bytes(&intent.market_id));       // u64 → 8 bytes
    vector::append(&mut buf, intent.trader);                          // 32 bytes (Sui address)
    vector::append(&mut buf, vector::singleton(intent.outcome_index)); // u8 → 1 byte
    vector::append(&mut buf, bcs::to_bytes(&intent.collateral_in));   // u64 → 8 bytes
    vector::append(&mut buf, bcs::to_bytes(&intent.max_slippage));    // u64 → 8 bytes
    vector::append(&mut buf, bcs::to_bytes(&intent.nonce));           // u64 → 8 bytes
    buf
}
```

```
// EVM side (decode hybrid-encoded payload)
function decodeBuyIntent(bytes memory payload) internal pure returns (BuyIntent memory) {
    require(payload.length >= 89, "Invalid BuyIntent payload length");
    uint256 offset = 0;

    uint256 marketId = uint256(bytes8(bytes(payload)[offset:offset+8]));
    offset += 8;

    address trader = address(bytes20(bytes(payload)[offset:offset+32])); // truncate 32→20 for EVM
    offset += 32;

    uint8 outcomeIndex = uint8(bytes1(bytes(payload)[offset]));
    offset += 1;

    uint256 collateralIn = uint256(bytes8(bytes(payload)[offset:offset+8]));
    offset += 8;

    uint256 maxSlippage = uint256(bytes8(bytes(payload)[offset:offset+8]));
    offset += 8;

    uint256 nonce = uint256(bytes8(bytes(payload)[offset:offset+8]));

    return BuyIntent(marketId, trader, outcomeIndex, collateralIn, maxSlippage, nonce);
}
```

Tradeoffs:

- **Pro:** No external libraries; deterministic encoding.
- **Con:** Manual serialization per struct; brittle if field order changes.

Option 2: Off-chain relay with re-encoding

A **relay service** (Go/Rust) fetches VAAs from the Guardian network, decodes BCS payloads, re-encodes to ABI, and submits to the EVM adapter.

Pro: Clean on-chain contracts; Move and Solidity use native encoding.
Con: Adds off-chain infrastructure; relay becomes a trust dependency (mitigated by making relay code open-source and permissionless).

Option 3: On-chain BCS decoder library (future)

Deploy a Solidity library that parses BCS-encoded bytes. Requires significant engineering and gas optimization.

Recommendation for Phase 3: Use **Option 1 (hybrid encoding)** for the initial Sui bridge. Plan migration to **Option 2 (relay)** when message volume justifies the infrastructure.

4.2 Canonical wire format per message type

Message Type	Sui → EVM (BCS or hybrid)	EVM → Sui (ABI-encoded, BCS-parsed on Sui)
BuyIntent	`market_id: u64, trader: [u8;32], outcome_index: u8, collateral_in: u64, max_slippage: u64, nonce: u64`	Same struct, abi.encode → BCS decode
SellIntent	`market_id: u64, trader: [u8;32], outcome_index: u8, shares_in: u64, min_payout: u64, nonce: u64`	Same
FillConfirmation	`intent_message_id: [u8;32], market_id: u64, trader: [u8;32], outcome_index: u8, shares_out: u64, execution_price: u64`	Same
ResolutionBroadcast	`market_id: u64, winning_outcome: u8, resolved_at: u64`	Same
RefundNotice	`intent_message_id: [u8;32], market_id: u64, trader: [u8;32], refund_amount: u64, reason: u8`	Same

Each field is big-endian; addresses are zero-padded to 32 bytes (Sui native) or truncated to 20 bytes (EVM).

---

5. Value legs: moving USDC Sui ↔ EVM

5.1 Circle CCTP integration (native USDC)

Sui supports **native Circle USDC** and is a CCTP-supported chain (as of 2025). Use **Circle CCTP** for the collateral leg:

Sui → EVM (BUY flow):

1. User locks `Coin<USDC>` in the Sui MarketProxy.
2. MarketProxy calls **Sui CCTP TokenMessenger** to burn USDC.
3. Off-chain: Circle's attestation service observes the burn, signs an attestation.
4. EVM side: a keeper (or the user) submits the attestation to the EVM CCTP MessageTransmitter, which mints native USDC on the destination chain.
5. HomeMarketGateway uses the minted USDC to execute the buy on the AMM.

EVM → Sui (REFUND or REDEMPTION flow):

1. HomeMarketGateway burns USDC via EVM CCTP.
2. Circle attestation service observes, signs.
3. Sui side: submit attestation to Sui CCTP, which mints native USDC back to the user's address.

5.2 When to use Wormhole Token Bridge vs CCTP

Scenario	Use CCTP	Use Wormhole Token Bridge
Collateral is USDC	✓ (native burn/mint, lower fee)	Only if CCTP unavailable on one side
Collateral is non-USDC	✗ (CCTP only supports USDC)	✓ (lock-and-mint wrapped tokens)
Sui ↔ Base/Ethereum/Polygon	✓ (all support CCTP)	Fallback if CCTP down
Sui ↔ BSC	Confirm (BSC CCTP TBD as of 2026)	✓ (Wormhole Token Bridge exists)

Integration note: The Sui `wormhole_adapter` module should expose both paths:

```
public entry fun send_buy_intent_with_cctp(
    // burn USDC via Sui CCTP, send Wormhole intent message
);

public entry fun send_buy_intent_with_wormhole_bridge(
    // lock USDC in Wormhole Token Bridge, send Wormhole intent message with token transfer
);
```

The frontend chooses the appropriate function based on route configuration.

5.3 Coordination: CCTP + Wormhole dual-leg timing

The **intent message** (via Wormhole) and the **USDC transfer** (via CCTP) are **independent legs** with different finality times:

- **Wormhole VAA:** available in ~seconds after Guardian observation (13-of-19 consensus).
- **CCTP attestation:** available in ~seconds after Circle's attestation service observes the burn.

Arrival order is nondeterministic. The EVM `HomeMarketGateway` must handle both cases:

- **Case 1:** Wormhole VAA arrives first → queue the intent, wait for CCTP USDC mint.
- **Case 2:** CCTP USDC mints first → hold in escrow, wait for Wormhole VAA, then execute.

Implementation: the gateway maintains a `pendingIntents` mapping keyed by `intentNonce`, tracking which leg has arrived. When both arrive, execute the AMM buy.

---

6. Chainlink CCIP mapping — architectural equivalence

The Sui bridge uses **Wormhole** as the concrete transport but maintains the **same architectural layering** as the EVM bridge's **Chainlink CCIP-inspired design**. This table explicitly maps Wormhole concepts to CCIP concepts to the Justify abstraction:

Justify Abstraction	Chainlink CCIP Equivalent	Wormhole Equivalent
Bridge transport	Chainlink CCIP messaging	Wormhole core bridge
Message attestation	Committing DON (builds Merkle root)	Guardian network (signs VAA)
Message verification	Executing DON (verifies proof, delivers)	Relayer submits VAA; core contract verifies Guardian signatures
Chain identifier	CCIP chain selector (uint64)	Wormhole chain ID (uint16)
Route / Lane	CCIP lane (source→dest pair)	Wormhole route (source→dest chain ID pair)

Message ID	CCIP messageId (bytes32, returned by ccipSend)	VAA hash (bytes32, computed after Guardian signing)
Emitter authentication	CCIP sender allowlist (per OffRamp)	Wormhole emitter allowlist (per adapter)
Replay protection	CCIP sequence numbers + nonce	VAA hash consumed flag + sequence per emitter
Finality enforcement	CCIP `consistencyLevel` (source finality)	Wormhole `consistencyLevel` (source finality)
Risk management layer	CCIP Risk Management Network (RMN) — blessing/cursing	Justify circuit breaker + exposure caps (no Wormhole-native RMN; protocol-layer control)
Value transfer	CCIP token pools (lock/mint or burn/mint)	Wormhole Token Bridge (lock/mint) or Circle CCTP (burn/mint native USDC)
Off-chain infrastructure	Chainlink DON nodes	Wormhole Guardian nodes + Guardian gossip network

6.1 Architectural parity

Both CCIP and Wormhole implement a **commit-then-execute pattern**:

- **CCIP:** Committing DON commits a Merkle root on-chain; Executing DON delivers individual messages with proofs.
- **Wormhole:** Guardians sign VAAs off-chain (commit); relayers/users submit VAAs on-chain (execute).

Justify's `IBridgeAdapter` abstracts both: `sendMessage()` initiates, `receiveMessage()` delivers, and the adapter handles the transport-specific commit/execute details.

6.2 Why Wormhole for Sui, CCIP for EVM

- **Wormhole:** production-proven for Sui↔EVM; Sui native integration; 19 Guardians including established validators.
- **CCIP:** mature EVM↔EVM lanes (Ethereum, Base, Polygon, etc.); Chainlink DON security model; native LINK fee model.

Both are wrapped behind `IBridgeAdapter`, so Justify's application layer (MarketProxy, HomeMarketGateway) is **transport-agnostic**. A future where Chainlink CCIP supports Sui, or Wormhole supports Arc, can be accommodated by deploying a different adapter without redesigning the core contracts.

---

7. Idempotency and ordering

7.1 Replay protection

Sui side:

- `AdapterState.consumed_vaas: Table<vector<u8>, bool>` — each incoming VAA hash is checked before processing and marked consumed after.

EVM side:

- `WormholeBridgeAdapter.consumedVAAs: mapping(bytes32 => bool)` — same pattern.

Guarantee: Each VAA is processed **at most once** per chain. If a VAA is submitted twice (user error, reorg, relayer retry), the second attempt reverts with `VAAReplayed`.

7.2 Ordering per trader

Each `BuyIntent` includes a **per-trader nonce**. The home `HomeMarketGateway` enforces:

```
mapping(address => mapping(uint64 => uint256)) public traderNonce;

function _validateIntent(BuyIntent memory intent, uint64 sourceChain) internal {
    require(
        intent.nonce == traderNonce[intent.trader][sourceChain] + 1,
        "Nonce out of order"
    );
    traderNonce[intent.trader][sourceChain] = intent.nonce;
}
```

Result: Intents from the same trader on the same source chain execute **in order** (nonce 1, then 2, then 3, ...). Intents from different traders or different chains may interleave.

7.3 Wormhole sequence numbers

Each Wormhole emitter (e.g., the `wormhole_adapter` module on Sui) maintains a **monotonic sequence number** per message. The core bridge increments it on every `publish_message()` call.

Use case: Off-chain indexers and relayers can track sequences to detect missed messages or ensure complete delivery.

Note: Wormhole sequences are **per emitter**, not global. If Justify deploys multiple Sui market contracts that each emit messages, each has its own sequence counter.

7.4 Effectively-once semantics

- **At-least-once delivery:** Wormhole Guardians will re-sign a VAA if requested (e.g., after a reorg), and relayers may submit the same VAA multiple times.
- **Idempotency (on-chain):** The adapter's `consumedVAAs` map makes duplicate VAAs a no-op → **effectively-once execution**.

---

8. Open questions and Wormhole Sui SDK confirmation

Topic	Status
Wormhole Sui SDK	Confirm module paths (`wormhole::state`, `wormhole::publish_message`, `wormhole::vaa`) from Wormhole Sui SDK docs. As of 2026, the SDK is under active development; API may differ.
Consistency level	Confirm Sui's finality level (e.g., 15 = full finality) from Wormhole docs. Sui uses Mysticeti consensus (sub-second finality); typical consistency level is lower than Ethereum.
Emitter address canonicalization	Wormhole uses 32-byte addresses; Sui addresses are 32 bytes (native), EVM are 20 bytes (left-padded to 32). Confirm padding convention from Wormhole SDK.
CCTP on Sui	Confirm Circle CCTP availability on Sui mainnet (testnet support confirmed as of 2025). If unavailable, fall back to Wormhole Token Bridge for USDC.
BCS↔ABI decoder	Production-quality BCS→ABI decoder library in Solidity does not exist as open-source as of 2026. Hybrid encoding (§4.1 Option 1) or off-chain relay (Option 2) are the paths forward.
Guardian set upgrades	Wormhole Guardians rotate periodically; VAAs include `guardianSetIndex`. Confirm the adapter tracks the active set index from the core bridge state.

Action for implementer: Validate all Wormhole SDK calls against the official Wormhole Sui SDK documentation before deploying to testnet.

---

9. Forward links

- **documentation_bridge/01-bridge-adapter-interface.md** — the `IBridgeAdapter` seam this Wormhole adapter implements.
- **documentation_bridge/02-chainlink-ccip-adapter.md** — the EVM-side CCIP adapter; this document's architectural parallel.
- **02-sui-bridge-overview.md** — the Sui bridge's home/proxy split and Chainlink-CCIP-inspired layering.
- **04-sui-bridge-security.md** — security model: Guardian trust, exposure caps, circuit breaker.
- **01-move-market-contracts.md** — the Sui-native prediction market stack this bridge connects to EVM.

---

10. Summary

This document defines the **Wormhole adapter and Sui↔EVM interop layer** for Justify's Sui bridge:

1. **Wormhole primer:** Guardian network, VAAs, core bridge per chain, sequence numbers, off-chain commit pattern.
2. **Sui Move adapter:** `wormhole_adapter` module publishes BCS-encoded BuyIntent messages, verifies incoming FillConfirmation VAAs, enforces emitter allowlisting and replay protection.
3. **EVM Solidity adapter:** `WormholeBridgeAdapter` implements `IBridgeAdapter` (same seam as CCIP), parses VAAs via Wormhole core, decodes BCS↔ABI payloads, forwards to `HomeMarketGateway`.
4. **Message encoding:** hybrid BCS/ABI format for MVP; path to off-chain relay for production.
5. **Value legs:** Circle CCTP for native USDC burn/mint (preferred); Wormhole Token Bridge fallback for non-USDC or unsupported lanes.
6. **Chainlink CCIP mapping:** explicit table mapping Guardian network ↔ DON, VAA ↔ CCIP commit, Wormhole chain ID ↔ CCIP selector — the adapter implements the **same Chainlink-CCIP-inspired IBridgeAdapter seam**, just with Wormhole as transport.
7. **Idempotency:** VAA hash consumed flags, per-trader nonces, sequence tracking → effectively-once execution.

The result: Justify's Sui markets bridge to EVM markets with the same security model, guaranteed-bounds fills, and transport abstraction as the EVM↔EVM bridge, extending the "chain is a parameter" thesis to non-EVM ecosystems.

File written: `/data/PROJECTS/PolyMarket/polymarket/documentation_sui/03-wormhole-adapter-and-interop.md` (316 lines)

Sui Bridge — Security and Risk Management

Document: 04-sui-bridge-security.md **Status:** Designed (separate project — Sui-based PolyMarkets + Sui bridge) **Related:** README.md | 02-sui-bridge-overview.md | 03-wormhole-adapter-and-interop.md | EVM analog: ../documentation_bridge/06-security-and-risk-management.md

This document is the Sui counterpart of the EVM bridge's security model. It keeps the **same Chainlink-CCIP-inspired defense-in-depth philosophy** as the EVM set, but adapts every layer to the **EVM↔Sui boundary** and to **Wormhole** as the concrete transport. Where the EVM bridge leans on a Chainlink CCIP lane (and its Risk Management Network), the Sui bridge leans on the **Wormhole Guardian network** — a different trust model that this document analyzes honestly.

> ***Honesty note.*** *Several Wormhole parameters cited below (Guardian quorum, Global Accountant / governor limits) change over time and across mainnet vs testnet. Values are given as "confirm current value" where they are not safe to hard-code. Nothing here is deployed; this is the target safety design.*

---

1. Trust model and attack surface

1.1 The dependency stack

A cross-chain position opened from Sui (Sui-as-proxy, EVM home — the recommended initial topology, see 02) depends on the integrity of **five layers**:

```
┌──────────────────────────────────────────────────────────┐
│  EVM Home Chain Safety & Liveness                        │  ← must finalize, not reorg
├──────────────────────────────────────────────────────────┤
│  EVM Home Market Contracts (MarketAMM, OracleResolver)   │  ← must price / settle correctly
├──────────────────────────────────────────────────────────┤
│  Wormhole transport (Guardian network + VAAs + relays)   │  ← must attest messages honestly,
once
├──────────────────────────────────────────────────────────┤
│  Sui MarketProxy (Move): lock / mint receipt / redemption    │  ← must escrow Coin<USDC>, honor
settlement
├──────────────────────────────────────────────────────────┤
│  Sui Chain Safety & Liveness                             │  ← must finalize user txs, not
equivocate
└──────────────────────────────────────────────────────────┘
```

The honest statement (same shape as whitepaper §5.2): a Sui-proxied position is **as safe as the weakest of (EVM home chain, Wormhole transport, Sui chain)**. A single-chain Sui market depends on one chain's safety; a bridged position inherits the risk of **two chains plus the Guardian set**.

1.2 Added attack surface vs single-chain — and vs an EVM-native CCIP lane

Vector	Single-chain Sui market	Sui↔EVM bridged (Wormhole)	Note vs EVM-native CCIP lane
Chain liveness	1 chain	2 chains (Sui + EVM home); either halting freezes *new* intents (redemptions stay pull-able)	Same 2-chain exposure
Reorg / equivocation	1 chain	2 chains; finality differences (Sui Mysticeti vs EVM finality) widen the replay window	Comparable

Transport trust	none	**Wormhole Guardian quorum** (m-of-n multisig of named operators)	**CCIP's RMN is an independent oracle network + DON, not a fixed multisig** — a different, arguably broader trust base. This is the main trust delta of choosing Wormhole for Sui today.
Contract bug	market objects	market + proxy + adapter (Move) + EVM home gateway + EVM adapter	More code, two languages (Move + Solidity)
Encoding	none	**BCS (Sui) ↔ abi (EVM)** mismatch is a new, bridge-specific failure class	Not present intra-EVM
Censorship/MEV	Sui validators	Sui + EVM validators + Wormhole relayers	One extra relay point

Why we accept the Guardian-set trust delta: Sui has no mature Chainlink CCIP lane today, and Wormhole is the established general-message bridge with first-class Move/Sui support. The architecture keeps the transport **pluggable behind the Move adapter seam** (03), so a future Sui CCIP lane can replace Wormhole without redesigning the application layer — at which point this trust delta closes.

---

2. Threat catalog with mitigations

#	Threat	Layer	Mitigation
1	**Forged / invalid VAA** (fake settlement or fill)	Transport	Verify every VAA against the on-chain Guardian set via the Wormhole core contract before acting; reject on signature/quorum failure. The adapter never trusts payload bytes that did not arrive inside a verified VAA.
2	**Guardian-set compromise** (≥ quorum of Guardians collude/keys stolen)	Transport	Cannot be fully mitigated at the app layer — this is the irreducible Wormhole trust assumption. Bound the damage with **per-route exposure caps** (§6) and the **circuit breaker** (§4) so a single forged batch cannot drain more than a route's cap. Monitor Guardian liveness/governance (§7).
3	**VAA replay** (re-submitting a valid VAA to double-credit)	Transport / Proxy	On-chain **consumed-VAA set**: store each redeemed VAA hash (Wormhole `vaa::digest`) in a shared object; reject if already present. Per-emitter **sequence numbers** enforce ordering. Effectively-once = at-least-once delivery + on-chain idempotency.
4	**Sui reorg / equivocation**	Sui chain	Wait for Sui **finality (checkpoint commitment)** before publishing a cross-chain intent; the proxy only emits the Wormhole message after the locking tx is final. Symmetrically, the EVM side waits for its own finality before acting on a Sui-origin VAA.
5	**Move object-ownership bug** (escrow drained / receipt minted without lock)	Proxy (Move)	Resource-typed escrow: locked collateral lives as a `Balance<USDC>` field inside the shared `Market`/`Proxy` object; it cannot be copied or dropped (Move resource safety). Receipts mint **only** in the same function that took custody, guarded by the conservation invariant (§ below). Capability-gated admin paths. Audit + formal-spec the escrow module.
6	**Emitter spoofing** (a different contract claims to be the home gateway)	Transport / Proxy	**Emitter allowlist per route**: the proxy accepts VAAs only from the registered `(wormhole_chain_id, emitter_address)` of the EVM home adapter; any other emitter is rejected. Symmetric allowlist on the EVM side for the Sui emitter.

7	**BCS ↔ abi deserialization mismatch** (encoding drift corrupts a payload)	Encoding	A **frozen wire spec** (see 03) with a `version` byte and explicit per-field width; cross-language round-trip test vectors in CI; reject unknown versions. A mismatch fails closed (revert/abort), never silently mis-credits.
8	**Liquidity drain via a bad/poisoned route**	App	Per-route **exposure cap** (§6) bounds the most a single route can owe; route registration is capability-gated; new routes start with a low cap and ramp.
9	**Stuck intents / griefing** (locked collateral, no fill/refund)	App	Every BuyIntent carries a deadline; past it the user can **pull a refund** of locked Coin on Sui without any counter-message (the home side, if it filled, reconciles via its own confirmation). Refund path is never pausable.
10	**Oracle / resolution manipulation**	Home	Resolution happens **once on the EVM home** via `OracleResolver` with the public oracle-proof commitment, then propagates to Sui as a `ResolutionBroadcast` VAA. No new oracle attack surface is introduced on Sui — the Sui side only *applies* a verified home resolution; it never decides outcomes.
11	**Relayer withholding** (a VAA is signed but not delivered)	Transport	VAAs are **pull-redeemable**: anyone (the user, a Justify relayer, a watchtower) can submit the VAA to the destination. Withholding delays, it cannot censor, because the signed VAA is public on the Guardian gossip layer.
12	**Global Accountant / governor stall** (Wormhole rate-limits a large transfer)	Transport	Expected behavior, not an attack: surface the delay in the UI; the value leg completes when the governor window clears. Keep per-trade sizes within the governor limit where known (confirm current limits).

---

3. The Chainlink RMN parallel — defense-in-depth inspiration

3.1 What Chainlink CCIP provides (the model we inherit conceptually)

Chainlink CCIP layers an **independent Risk Management Network (RMN)** beneath the DON: a separate set of nodes, with separate code and operators, that **blesses** (approves) committed message roots and can **curse** (halt) a lane on detecting an anomaly — a second, independent validation of the same messages. CCIP also enforces **per-lane rate limits** (token-pool caps) that bound throughput.

3.2 Wormhole's analog — and how it differs

Concern	Chainlink CCIP	Wormhole (Sui transport)
Message attestation	DON commits a Merkle root	**Guardian network** signs a VAA (quorum, e.g. ~13-of-19 — *confirm current quorum*)
Independent second check	**RMN** blesses/curses, separate node set	The Guardian quorum **is** the attestation; there is no fully separate "second network" — closer to a single m-of-n multisig of reputable operators
Throughput limit	per-lane rate limits	**Global Accountant** + **governor** caps per chain/asset (*confirm current params*)
Trust shape	oracle network + independent risk net	named-operator multisig quorum

Honest comparison. CCIP's RMN gives a *structurally independent* second validation; Wormhole's security rests on the **diversity and honesty of the Guardian set** plus the Accountant's rate limits. Wormhole's is a narrower trust base than RMN's "two independent networks" design. We accept it because it is what ships on Sui today, and we **compensate at the application layer** (§3.3) and keep the transport swappable.

3.3 Justify's RMN-inspired, application-layer guards (same philosophy as EVM)

Exactly as the EVM bridge does, the Sui bridge does **not** rely on the transport alone. On top of Wormhole it adds:

1. **Per-route exposure caps** (§6) — bound the blast radius of *any* transport failure, including a Guardian-set compromise.
2. **Emitter/adapter allowlist per route** (§2 #6) — only the registered home gateway can settle a given proxy market.
3. **Circuit breaker** (§4) — pause *new* intents on anomaly, never block exits.
4. **On-chain idempotency** (consumed-VAA set, §2 #3).

This is the Chainlink-CCIP-inspired **defense-in-depth** stance: transport-level risk controls (Guardians + Accountant ≈ RMN + rate limits) **plus** independent application-level controls we own and can tune per route.

---

4. The circuit breaker — pause new intents, never block redemptions

4.1 Design invariant

> *The breaker may pause **new cross-chain intents** in either direction. It must **never** block **redemption / exit** of an already-settled or refundable position on either chain.*

A user who has locked collateral or holds a wrapped receipt must always be able to get out — via the home settlement they are owed, or via the deadline refund.

4.2 What trips it

- Cross-route **refund rate** exceeds a threshold (e.g. > 5% of intents in a window) — signals encoding/route trouble.
- **Outstanding owed** on a route approaches its exposure cap (§6).
- **Confirmation/fill failures** spike (home side rejecting Sui-origin intents).
- **VAA verification failures** spike (possible forged-VAA probing).
- **Guardian liveness** alert (quorum at risk) — confirm via Wormhole status.

Tripping can be automatic (on-chain counters crossing a threshold) or manual (BridgeAdmin, §5).

4.3 Move-side implementation

A shared `BridgeConfig` object holds the breaker state; the entry that opens a new intent checks it, the redemption entry does not.

```
module justify_bridge::config {
    use sui::object::UID;

    /// Shared object; one per deployment. Gated by AdminCap for mutation.
    public struct BridgeConfig has key {
        id: UID,
        paused_intents: bool,      // breaker: blocks NEW intents only
        // per-route caps & outstanding tracked in a Table keyed by route id
    }

    /// Called by the proxy BEFORE locking collateral / publishing a Wormhole msg.
    public fun assert_intents_live(cfg: &BridgeConfig) {
        assert!(!cfg.paused_intents, E_INTENTS_PAUSED);
    }

    // NOTE: there is deliberately NO `assert_*` guard on the redemption /
```

```
    // refund path — exit is always permitted. The breaker is one-directional.
}
```

The redemption and deadline-refund entries in the proxy **never call** `assert_intents_live`, structurally guaranteeing the exit invariant.

4.4 Recovery / reset

Un-pausing is capability-gated (BridgeAdminCap) and should follow a checklist: root cause identified, outstanding reconciled, caps reviewed. Optionally a timelock on re-enabling a route after a trip.

---

5. Governance and access control (Move capability pattern)

5.1 Capabilities replace role mappings

EVM `AccessControl.sol` uses role → address mappings. Sui/Move uses **capability objects**: holding the object *is* the authorization (no global mapping to check, no role-grant griefing). Building on 01:

Capability (owned object)	Authorizes	Held by
`AdminCap`	top-level admin of the markets package	protocol multisig
`BridgeAdminCap`	allowlist emitters/routes; set exposure caps; trip/reset breaker	bridge governance multisig
`ResolverCap`	apply a verified `ResolutionBroadcast` on Sui	the bridge adapter (delegated), not a human

Capabilities are transferable to a multisig / governance object; revocation = move the cap to a burn/escrow object. No single EOA holds god power.

5.2 Who allowlists what

- **Routes & emitters:** `BridgeAdminCap` registers each `(wormhole_chain_id, emitter_address) → route` pair. An unregistered emitter's VAA is rejected (§2 #6). New routes start paused / low-cap and ramp.
- **Exposure caps:** `BridgeAdminCap` sets and adjusts per-route caps (§6).
- **Breaker:** `BridgeAdminCap` trips/resets (§4).

5.3 Upgrade philosophy — and Sui's package-upgrade reality

Whitepaper §7: no upgradeable god-contract; fix a bad parameter by **redeploying a market**, not by mutating the protocol. Two Sui-specific honesty points:

- Sui packages can be **immutable** or **upgradeable** (UpgradeCap). We prefer **immutable market logic** with parameters in mutable *shared objects* (caps, allowlists, breaker) — so behavior is tuned via data, not code, and the code surface is frozen and auditable.
- Where upgradeability is genuinely needed, the `UpgradeCap` is held by the same governance multisig and its policy (e.g. `additive` / `dep-only`) should be restricted; document any upgradeable package explicitly. Confirm the chosen policy against the current Sui framework version.

---

6. Per-route exposure caps — bounding the blast radius

6.1 Threat model

Caps exist so that *no single transport or route failure* — including a Guardian-set compromise — can cost more than a bounded amount. The cap is the hard ceiling on **outstanding collateral owed across a route** at any time.

6.2 How exposure is tracked

For each route, the proxy maintains `outstanding` in the `BridgeConfig` table:

- **+** when a BuyIntent locks collateral and is in-flight to the home market.
- **–** when a FillConfirmation or RefundNotice for that route is applied.
- A new intent **aborts** if `outstanding + amount > cap`.

```
public fun reserve_route_capacity(cfg: &mut BridgeConfig, route: u16, amount: u64) {
    let used = current_outstanding(cfg, route);
    assert!(used + amount <= route_cap(cfg, route), E_ROUTE_CAP_EXCEEDED);
    set_outstanding(cfg, route, used + amount);
}
```

6.3 Suggested parameters (illustrative — tune in production)

Route	Transport	Suggested initial cap
Sui → EVM home (Arc/Base)	Wormhole	start small (e.g. 250k USDC), ramp with confidence
EVM home → Sui (settlement)	Wormhole	settlement is pull-based; cap the *new-intent* direction, not exits

Ramp caps as the route accrues a clean track record; never set a cap above the **Wormhole governor limit** for the asset (confirm current limit) — the governor would throttle anyway.

6.4 Relation to Wormhole governor limits

The Wormhole **Global Accountant / governor** already rate-limits large transfers per chain/asset. Justify's per-route caps are **complementary, not redundant**: the governor is global and protocol-wide; our caps are **per-market-route and tunable by us**, and trip our own breaker — giving an independent, faster, application-owned control surface.

---

7. Operational runbook (stub)

7.1 Monitoring signals

- **VAA latency** Sui→EVM and EVM→Sui (publish → redeem); alert on tail spikes.
- **Guardian liveness / quorum health** (confirm via Wormhole status feeds).
- **Refund rate** per route (breaker input, §4.2).
- **Outstanding vs cap** per route (approaching-cap alert).
- **VAA verification failures** (forged-VAA probing).
- **Consumed-VAA set growth** vs expected message volume (replay attempts).

7.2 Incident response

1. **Trip the breaker** (`BridgeAdminCap`) — pauses new intents both directions; exits remain open (§4.1).
2. **Diagnose** — encoding mismatch? emitter anomaly? Guardian alert? home-side rejection? Use the monitoring signals + the VAA hash to trace a specific message.
3. **Contain** — if one route is implicated, lower its cap to 0 (route-level pause) while leaving healthy routes live.

7.3 Recovery

- **Replay redeemable VAAs** — signed VAAs are public; anyone can resubmit a withheld settlement VAA to unstick a position (§2 #11).
- **Pull-based exit** — users redeem settled receipts / claim deadline refunds with no dependency on the breaker state.
- **Reset** — un-pause per §4.4 after root cause + reconciliation.

---

8. Summary

The Sui bridge keeps the EVM bridge's **Chainlink-CCIP-inspired defense-in-depth**: rely on the transport's own risk layer — here the **Wormhole Guardian quorum + Global Accountant**, the analog of CCIP's **RMN + rate limits** — and **add independent application-layer guards** Justify owns: per-route exposure caps, an emitter/adapter allowlist per route, on-chain VAA idempotency, and a one-directional circuit breaker that pauses new intents but **never blocks exit**.

The honest trade-off of choosing Wormhole for Sui is a **narrower transport trust base** than a Chainlink CCIP lane's RMN. We bound that risk with caps + breaker and keep the transport **swappable behind the Move adapter seam**, so a future Sui CCIP lane closes the gap without an application rewrite.

> *Designed, not deployed. Wormhole quorum/governor parameters and Sui package- upgrade policy must be confirmed against current Wormhole and Sui framework versions before any deployment.*